SAMPLING THE FUTURE: MONTE-CARLO QUERY ESTIMATION FOR KV CACHE EVICTION

Anonymous authors
Paper under double-blind review

ABSTRACT

MV: how about we say something more catchy and very understandable for aanywone: in this paper, we move towardss asking "which memory will matter while answering?" rather than "which memory appeared important while reading?"

Most KV-cache eviction methods ask, in effect, *"which memory appeared important while reading the prompt?"* We instead ask, *"which memory will matter while answering?"* The difficulty is that future decoding queries are unknown when the cache is compressed. We address this by treating the future-query distribution as something to *sample* rather than guess. We introduce **LORE-KV** (***L**ookahead **O**utput-perturbation with **R**eliability-weighted **E**nsembles for **K**ey–**V**alue caches*), a training-free KV-cache eviction method. LORE-KV samples M short continuations from the frozen model, uses their query states as a Monte-Carlo estimate of future decoding queries, and scores each KV pair by its projected leave-one-out attention-output perturbation under this ensemble. Each trajectory receives a reliability weight based on branch atypicality, context uncertainty, and, optionally, disagreement with the other trajectories in query space. The lookaheads are discarded before final generation, requiring no training, auxiliary model, or modification of the base LLM. Across LongBench, RULER, and Needle-in-a-Haystack, LORE-KV remains consistently effective across models, tasks, and cache budgets, with its largest gains under aggressive compression; moreover, the gains saturate with only a few lookahead trajectories and increase with prompt-to-future query mismatch, showing that the improvement comes primarily from better future-query estimation rather than simply spending more test-time compute.

1 INTRODUCTION

Autoregressive large language models (LLMs) rely on key-value (KV) caching to reuse previously computed hidden states during decoding. While KV caching greatly accelerates generation, its memory and bandwidth cost grows with sequence length and batch size, making long-context inference increasingly expensive (Behnam et al., 2025; Li et al., 2025a; Łańcucki et al., 2026). KV-cache eviction reduces this cost by retaining only the KV pairs expected to be useful during generation. Existing methods estimate token importance using recency or attention sinks (Xiao et al., 2024), accumulated or prompt-end attention (Zhang et al., 2023; Liu et al., 2023; Li et al., 2024), key geometry (Devoto et al., 2024; Park et al., 2026), or learned retention signals (Moschella et al., 2026; Bui et al., 2026).

A central limitation of existing KV-cache eviction methods is that they often estimate token importance using prompt-side queries, even though the retained cache is later used by future decoding queries. Lookahead Q-Cache (Wang et al., 2025b) partially addresses this by using a single surrogate response, while learned methods such as LookaheadKV estimate future utility with trained predictors (Ahn et al., 2026; Moschella et al., 2026; Bui et al., 2026); however, a single surrogate captures only one possible future, and learned approaches require extra training and model-specific parameters. At the same time, attention alone is not a complete measure of deletion impact, because tokens with similar attention can contribute differently through their value vectors (Guo et al., 2024). Output-aware methods such as AnDPro and LaProx improve the scoring criterion (Geng et al., 2026; Mai & Kim, 2026), but still generally rely on prompt-side queries. Thus, prior work improves either

the future-query estimate or the deletion score, but not both together over multiple plausible future continuations.

We propose **LORE-KV**, short for ***L**ookahead **O**utput-Perturbation with **R**eliability-weighted **E**nsembles*, a training-free KV-cache eviction method that *samples the future* instead of relying on prompt-side queries or a single surrogate continuation. LORE-KV generates multiple short continuations from the frozen model, uses their query states to approximate plausible future decoding queries, and scores each KV pair using projected leave-one-out output perturbation. It combines response-side lookahead from Lookahead Q-Cache (Wang et al., 2025b) with projected output-aware scoring from AnDPro (Geng et al., 2026), extending both with multiple trajectories and reliability-weighted aggregation Unlike learned future-utility methods (Ahn et al., 2026; Moschella et al., 2026; Bui et al., 2026; Yang et al., 2026), LORE-KV requires no offline training or auxiliary predictor, adds only a small one-time lookahead cost, and empirically gains most when prompt-side and future queries differ, supporting better query estimation rather than extra test-time compute as the main source of improvement.

MV: The strongest novelty seems to be the combination of future-query estimation with projected deletion scoring. Could we state somwhere explicitly that response-side lookahead is "inherited" from Lookahead Q-Cache and projected output-aware scoring from AnDPro, then isolate empirically what multi-trajectory confidence weighting adds beyond them?

Contributions. Our contributions are threefold. **First**, we identify and empirically characterize the *query-reference mismatch* in KV-cache eviction, showing that prompt-side queries can be poor proxies for the future decoding queries that consume the retained cache. **Second**, we introduce LORE-KV, a training-free combination of multiple sampled future-query trajectories, projected leave-one-out deletion scoring, and reliability-weighted aggregation. Unlike learned future-utility methods, LORE-KV requires no offline training, auxiliary predictor, or model-specific learned parameters, while directly adapting its query estimate to each input at test time. **Third**, we evaluate LORE-KV across LongBench, RULER, and Needle-in-a-Haystack on two LLM backbones and multiple cache budgets, with ablations showing that its gains primarily arise from improved future-query estimation rather than additional test-time compute.

MV: suggestion: somehow, what we do is: training-free combination: multiple sampled future-query trajectories + projected leave-one-out scoring + confidence aggregation.
at the same time, there were combinations that were trained, so naturally people will think, "Well, with training I can do better." I think we should only then defend it. What is the benefit of non-training: computational efficiency, collapse avoidance, or whatever, and back to it empirical evidence

2 METHODOLOGY

LORE-KV has three stages: (i) M short continuations form a Monte-Carlo estimate of future decoding queries (Section 2.2); (ii) each trajectory receives a reliability weight based on likelihood, entropy, or ensemble disagreement (Section 2.3); and (iii) a projected counterfactual deletion score measures the effect of evicting each KV pair under the weighted future queries (Sections 2.4–2.5). Algorithm 1 summarizes the method, Table 5 lists the hyperparameters, and Appendices B and C provide implementation details and ablations.

2.1 PROBLEM SETUP

Let f_θ denote a frozen pretrained transformer language model, whose parameters θ remain fixed throughout. Given a prompt $X = (x_1, \ldots, x_n)$, a single prefill pass produces the cached keys and values $\mathbf{K}^{l,h} = \{\mathbf{k}_1^{l,h}, \ldots, \mathbf{k}_n^{l,h}\}$ and $\mathbf{V}^{l,h} = \{\mathbf{v}_1^{l,h}, \ldots, \mathbf{v}_n^{l,h}\}$ for each layer $l \in \{1, \ldots, L\}$ and KV head $h \in \{1, \ldots, H\}$, together with the prompt-window queries $\mathcal{Q}_{\mathcal{W}}^{l,h}$. Let $\mathcal{W} = \{n-w+1, \ldots, n\}$ denote the prompt-end observation window of size w, and let $\mathcal{B}$ denote the optional structural-token positions (more details in Appendix B.1). We also retain the first prompt token, which commonly acts as an attention sink (Xiao et al., 2024). Thus, the protected and candidate sets are $\mathcal{P} = \{1\} \cup \mathcal{W} \cup \mathcal{B}$ and $\mathcal{C} = \{1, \ldots, n\} \setminus \mathcal{P}$, respectively, with only tokens in $\mathcal{C}$ scored for eviction.

2.2 TEST-TIME LOOKAHEAD QUERY CONSTRUCTION

Prompt-side queries may poorly represent the response-side queries used during generation. To reduce this mismatch, LORE-KV uses a small, fixed test-time budget before final decoding to sample M short lookahead continuations from the frozen model:

$$Y^{(m)} \sim f_\theta(\cdot \mid X), \qquad Y^{(m)} = \{y_1^{(m)}, \ldots, y_T^{(m)}\}, \qquad m = 1, \ldots, M, \tag{1}$$

where T is small. The continuations are used as temporary probes for estimating future decoding queries. To keep the reliability score correctly centered, each trajectory is evaluated under the same distribution from which it is sampled (Section 2.3). For each trajectory m, we store its lookahead query states as $\mathcal{Q}_{\text{look}}^{(m),l,h} = \left\{\mathbf{q}_1^{(m),l,h}, \ldots, \mathbf{q}_T^{(m),l,h}\right\}$, and combine them with the prompt-window queries for scoring:

$$\mathcal{Q}_{\text{score}}^{l,h} = \mathcal{Q}_{\mathcal{W}}^{l,h} \cup \left(\bigcup_{m=1}^{M} \mathcal{Q}_{\text{look}}^{(m),l,h} \right). \tag{2}$$

The M lookahead trajectories therefore provide response-side query samples for subsequent KV scoring. Figure 4 shows that $\mathcal{Q}_{\text{score}}^{l,h}$ better covers the true future queries than $\mathcal{Q}_{\mathcal{W}}^{l,h}$ alone, while Appendix B.2 relates this construction to prior surrogate-query methods.

2.3 PARTICLE RELIABILITY WEIGHTING: ENTROPY AND TRAJECTORY DISAGREEMENT

The lookahead trajectories can differ in reliability, so LORE-KV assigns each a weight. Let $u \in 1, \ldots, T$ index the T lookahead steps, and let $h^{(m)}u = (X, y^{(m)}< u)$ denote the context of trajectory m at step u. Let $q(\cdot \mid h)$ be the sampling distribution. Raw log-probability mixes two effects: whether the sampled token is atypical for its local context and how uncertain that context is. We separate these channels with the centered score

$$c_u^{(m)} = \log q\Big(y_u^{(m)} \mid h_u^{(m)}\Big) + H\Big(q(\cdot \mid h_u^{(m)})\Big), \tag{3}$$

for which $\mathbb{E}[c_u^{(m)} \mid h_u^{(m)}] = 0$. With $\bar{c}_m = \frac{1}{T}\sum_u c_u^{(m)}$ and $\bar{H}_m = \frac{1}{T}\sum_u H(q(\cdot \mid h_u^{(m)}))$, trajectory reliability is

$$R_m = \beta\, \bar{c}_m - \gamma\, \bar{H}_m - \lambda_D\, \hat{D}_m, \tag{4}$$

where $\bar{c}_m$ measures branch atypicality relative to the uncertainty of the visited contexts, $\bar{H}_m$ penalizes high-uncertainty histories, and $\hat{D}_m$ is standardized disagreement between the trajectory's lookahead query states and those of the other particles. The definition, standardization, calibration, and finite-M edge cases of $\hat{D}_m$ are given in Appendix B.3. The reliability scores are converted to soft particle weights,

$$\omega_m = \frac{\exp(R_m)}{\sum_{r=1}^{M} \exp(R_r)}, \qquad \boldsymbol{\omega} = \mathrm{softmax}(\mathbf{R}) \in \Delta^{M-1}. \tag{5}$$

These weights control each trajectory's contribution to the future-query term in Section 2.5. Soft weighting is used rather than selecting a single trajectory because the measured reliability signals are only imperfect proxies for oracle trajectory utility (Appendix E.4).

We consider two settings of Eq. equation 4. ***LORE-KV/E*** uses $(\beta, \gamma, \lambda_D) = (1, 1.3, 0)$ and is the default. ***LORE-KV/D*** uses $(1, 1, 0.3)$ and additionally down-weights trajectories that are atypical in the query space used by the eviction score. The variants reuse the same trajectories and differ only in reliability coefficients, with equal decoding cost. Appendix B.3 provides details, and Table 10 reports single-factor controls.

2.4 CLOSED-FORM COUNTERFACTUAL DELETION EFFECT

With the scoring queries in hand, we now specify *what* is measured for each candidate token. For a scoring query $\mathbf{q}^{l,h}$ and candidate token $i \in \mathcal{C}$, the full-cache attention weight is $a_{q,i}^{l,h} =$

$\mathrm{Softmax}_i\Big((\mathbf{q}^{l,h})^\top \mathbf{k}_i^{l,h}/\sqrt{d}\Big)$, where d is the head dimension, and the corresponding full-cache attention output is $\mathbf{o}_q^{l,h} = \sum_{j=1}^{n} a_{q,j}^{l,h}\mathbf{v}_j^{l,h}$. We write $(a_{q,i}^{l,h}, \mathbf{o}_q^{l,h}) = \mathrm{Attn}(\mathbf{q}^{l,h}, \mathbf{K}^{l,h}, \mathbf{V}^{l,h})$ for this full-cache attention weight–output pair. If token i is removed and the remaining attention weights are renormalized, the post-deletion output becomes $\mathbf{o}_{q,-i}^{l,h} = \sum_{j\neq i} \frac{a_{q,j}^{l,h}}{1-a_{q,i}^{l,h}+\epsilon}\mathbf{v}_j^{l,h} = \frac{\mathbf{o}_q^{l,h} - a_{q,i}^{l,h}\mathbf{v}_i^{l,h}}{1-a_{q,i}^{l,h}+\epsilon}$, where ϵ prevents numerical instability. The corresponding leave-one-out deletion effect is therefore

$$\Delta\mathbf{o}_{q,i}^{l,h} = \mathbf{o}_q^{l,h} - \mathbf{o}_{q,-i}^{l,h} \approx \frac{a_{q,i}^{l,h}}{1 - a_{q,i}^{l,h} + \epsilon}\left(\mathbf{v}_i^{l,h} - \mathbf{o}_q^{l,h}\right). \tag{6}$$

Eq. equation 6 shows why attention alone is insufficient: a token is important only when it receives non-negligible attention and its value meaningfully differs from the current attention output.

Output-projection-aware causal score. Let $\mathbf{W}_{l,h}^O$ denote the output-projection block for head h in layer l. LORE-KV scores the residual-stream impact of deleting token i as

$$e_{q,i}^{l,h} = \left\|\mathbf{W}_{l,h}^O \Delta\mathbf{o}_{q,i}^{l,h}\right\|_2^2. \tag{7}$$

Unlike raw attention, this score measures the deletion effect after projection into the model stream. Further details are given in Appendix B.2. Figure 1 evaluates the resulting cache fidelity, while Figure 9 shows how the perturbation is distributed across layers and heads.

2.5 Core Score, Selection, and Compressed Cache

For each token i, we collect perturbation scores over the prompt-window queries, $\mathcal{E}_{i,\mathcal{W}}^{l,h} = \{e_{q,i}^{l,h} : q \in \mathcal{Q}_{\mathcal{W}}^{l,h}\}$, and each lookahead trajectory, $\mathcal{E}_{i,m}^{l,h} = \{e_{q,i}^{l,h} : q \in \mathcal{Q}_{\text{look}}^{(m),l,h}\}$. Using mean aggregation, $\rho(\mathcal{E}) = \mathrm{Mean}(\mathcal{E})$, with further details in Appendix C, the final token score is

$$S_i^{l,h} = \rho(\mathcal{E}_{i,\mathcal{W}}^{l,h}) + \lambda_L \sum_{m=1}^{M} \omega_m\, \rho(\mathcal{E}_{i,m}^{l,h}), \tag{8}$$

where λ_L governs how much the future-query information counts, and we stack the per-token scores of a head into the vector $\mathbf{S}^{l,h} = (S_i^{l,h})_{i\in\mathcal{C}}$. At $\lambda_L = 0$, LORE-KV collapses to a prompt-window output-perturbation method; as λ_L grows, the policy leans more heavily on the future queries. The lookahead term is not a cosmetic re-weighting of the prompt-side score: Figure 5 shows that roughly one in five of the tokens retained at $B = 128$ differs between the two rules, and Table 10 prices the λ_L=0 setting directly.

Two operators turn these scores into a compressed cache, and both reduce to their simplest form in the core method:

$$\{B_{l,h}\}_h = \mathrm{Alloc}(B_l, \{E_{l,h}\}_h; \tau), \qquad \mathcal{R}^{l,h} = \mathrm{Select}\big(\mathbf{S}^{l,h}, B_{l,h}\big). \tag{9}$$

For the core method, Alloc with $\tau = 0$ returns the uniform split $B_{l,h} = \lfloor B_l/H \rfloor$ of the per-layer budget B_l (reserved after protected tokens), and Select keeps the top-$B_{l,h}$ candidates by score. The final retained set and the compressed cache are

$$\mathcal{R}_{\text{final}}^{l,h} = \mathcal{P} \cup \mathcal{R}^{l,h}, \qquad \mathbf{K}_\star^{l,h}, \mathbf{V}_\star^{l,h} = \mathrm{Gather}\left(\mathbf{K}^{l,h}, \mathbf{V}^{l,h}, \mathcal{R}_{\text{final}}^{l,h}\right), \tag{10}$$

at which point the lookahead continuations are thrown away and final decoding runs on $(\mathbf{K}_\star, \mathbf{V}_\star)$. The switches $\tau > 0$ and $\lambda_k, \lambda_v > 0$ turn Alloc and Select into the optional plug-in rules of Appendix C, which are off throughout the core method; each recovers its default exactly at zero (Table 5).

3 Experiments

We follow the evaluation protocol of AnDPro (Geng et al., 2026), comparing all methods at matched cache budgets, backbones, and observation windows. We evaluate on **LongBench** (Bai

Table 1: LongBench results with Mistral-7B-Instruct-v0.2 across cache budgets. Baseline values are from Geng et al. (2026); – denotes runs in progress. LAQ++(8) is a compute-matched extension of Lookahead Q-Cache (Wang et al., 2025b) run under our implementation at $B \in \{128, 256, 512\}$ (not evaluated at $B = 1024$); it is reported for reference and excluded from the **/** highlighting.

	Single-Doc. QA			Multi-Doc. QA			Summarization			Few-shot			Synthetic		Code		
Method	NrtvQA	Qasper	MF-en	HotpotQA	2WikiMQA	Musique	GovReport	QMSum	MultiNews	TREC	TriviaQA	SAMSum	PCount	PRe	Lcc	RB-P	Avg.
Full Cache	26.63	32.99	49.34	42.77	27.35	18.77	32.87	24.24	27.10	71.00	86.23	42.96	2.75	86.98	55.33	52.87	42.51
Budget = 128																	
H2O	21.19	21.66	38.60	30.63	20.65	12.19	20.65	22.42	21.81	39.00	82.52	40.68	2.98	79.56	49.13	46.76	34.40
StreamingLLM	16.61	14.74	31.40	28.05	21.36	12.08	18.44	18.91	19.26	43.50	74.22	29.00	2.75	31.65	41.27	38.84	27.63
SnapKV	19.17	21.40	42.93	36.76	22.44	15.86	19.16	21.84	21.55	47.50	84.15	40.24	2.30	68.26	50.69	47.13	35.09
PyramidKV	20.16	21.77	43.55	36.78	23.12	14.39	19.53	22.03	21.47	51.00	84.62	40.24	2.79	70.77	50.57	46.53	35.58
Ada-KV	21.79	23.03	47.07	38.70	22.85	15.92	19.94	23.05	21.84	63.00	85.36	40.01	3.20	72.13	51.36	49.29	37.41
CriticalKV	21.55	23.27	46.67	38.95	24.95	16.85	20.83	23.14	22.16	65.00	85.53	40.02	**3.48**	77.81	53.58	50.98	38.42
AnDPro	24.84	25.14	47.33	39.79	23.99	17.34	20.40	**23.45**	21.94	67.50	85.59	40.62	3.19	76.94	53.77	51.55	38.96
LAQ++(8)	25.62	27.21	46.16	40.60	25.93	18.44	21.60	23.07	22.42	70.00	86.18	42.03	3.51	74.81	54.68	50.92	39.57
LORE-KV /E (ours)	24.05	26.85	**47.59**	42.19	**26.39**	**19.20**	21.48	22.46	**22.78**	69.50	**86.64**	41.08	3.44	84.37	**54.90**	51.56	40.28
LORE-KV /D (ours)	**25.14**	**27.94**	47.34	**42.26**	25.91	18.71	**21.54**	23.35	22.61	**70.00**	86.33	**41.24**	3.03	**84.51**	54.24	**52.03**	**40.39**
Budget = 256																	
H2O	21.54	22.92	42.56	31.07	22.53	13.76	22.52	22.40	23.09	40.50	84.20	40.77	**3.41**	86.10	50.98	48.17	36.03
StreamingLLM	17.93	16.01	33.36	30.71	21.30	10.08	20.66	19.47	22.89	53.50	73.59	29.22	3.00	27.77	42.30	39.87	28.85
SnapKV	22.37	23.74	48.13	38.56	22.43	15.66	21.91	23.13	23.15	61.50	85.45	41.42	3.09	84.54	53.22	50.24	38.66
PyramidKV	20.09	24.00	47.33	38.24	22.48	16.02	21.40	22.45	22.63	63.00	84.93	40.98	3.40	82.48	52.78	49.36	38.22
Ada-KV	23.02	25.70	49.23	40.18	24.76	17.43	22.17	23.21	23.48	67.50	85.58	41.67	2.99	84.65	55.07	52.17	39.93
CriticalKV	23.70	26.66	49.00	40.05	25.12	17.86	21.99	23.35	23.45	68.00	86.19	42.07	2.92	85.81	53.81	50.42	40.03
AnDPro	25.71	28.17	**49.31**	40.04	25.81	18.68	21.66	23.64	23.63	70.50	**86.59**	**42.60**	2.86	85.62	55.74	52.87	40.84
LAQ++(8)	25.23	29.16	49.24	41.70	26.85	18.62	23.73	23.69	23.38	70.50	86.24	42.54	3.36	86.11	55.59	52.49	41.15
LORE-KV /E (ours)	25.88	29.58	48.10	42.13	**26.83**	**18.90**	23.23	23.79	**24.07**	70.50	86.31	41.09	3.27	89.09	**56.15**	53.12	41.38
LORE-KV /D (ours)	**26.49**	**30.22**	47.96	**42.17**	26.56	18.63	**23.26**	**24.28**	23.98	**70.78**	86.14	41.18	3.04	**89.17**	55.78	**53.38**	**41.44**
Budget = 512																	
H2O	21.72	26.03	44.81	32.33	23.16	14.86	23.65	22.84	24.70	42.00	85.22	41.57	**3.40**	86.45	53.04	49.68	37.22
StreamingLLM	18.76	17.17	37.09	30.21	21.64	9.93	24.44	20.00	**25.57**	62.00	72.36	29.95	2.48	18.17	43.70	40.13	29.60
SnapKV	24.60	27.81	48.98	39.46	25.25	16.98	23.70	22.96	24.37	67.00	85.88	41.26	2.78	86.56	54.81	51.71	40.26
PyramidKV	23.23	27.94	48.87	40.50	24.36	16.74	23.22	23.16	24.37	67.00	85.73	41.74	3.16	85.67	54.16	50.34	40.01
Ada-KV	23.88	29.05	48.79	40.44	25.30	17.66	23.47	23.47	24.48	69.50	86.39	41.38	2.94	87.56	56.58	52.88	40.86
CriticalKV	23.83	28.67	49.13	40.18	26.18	18.19	23.52	23.57	24.28	70.00	86.39	42.16	2.82	87.55	56.51	52.64	40.98
AnDPro	24.87	30.99	**49.58**	41.34	27.01	18.44	23.40	23.76	25.01	71.00	**86.74**	**43.65**	2.83	86.23	**57.46**	53.61	41.62
LAQ++(8)	25.49	30.92	49.72	41.50	26.84	19.20	25.67	24.04	25.31	71.00	86.43	43.14	2.90	85.27	56.80	53.54	41.70
LORE-KV /E (ours)	25.58	**31.14**	48.98	41.82	**27.41**	**19.12**	25.46	23.97	25.05	71.00	86.49	41.92	2.95	88.20	56.85	53.82	**41.86**
LORE-KV /D (ours)	**25.98**	30.90	48.89	**41.85**	27.23	18.94	**25.48**	**24.30**	24.99	**71.18**	86.38	41.98	2.80	**88.25**	56.61	**53.99**	**41.86**
Budget = 1024																	
H2O	23.90	28.62	46.46	37.03	24.74	15.04	25.30	23.11	25.92	46.00	85.93	41.80	**3.24**	86.57	54.46	51.01	38.70
StreamingLLM	19.42	21.69	41.75	32.40	22.18	11.18	**27.13**	21.09	**26.59**	67.00	71.79	30.11	2.88	16.57	44.82	39.76	31.02
SnapKV	25.47	29.57	49.33	40.90	25.53	**19.01**	25.94	23.89	26.21	69.50	86.48	42.10	2.98	**88.56**	55.57	51.92	41.44
PyramidKV	24.21	29.86	48.93	40.75	25.05	18.77	25.73	24.06	25.65	68.50	86.31	42.25	2.97	87.17	54.75	52.10	41.07
Ada-KV	25.60	31.15	49.02	41.47	27.07	18.77	25.40	23.84	25.92	70.50	86.55	43.07	2.54	87.14	57.27	**53.85**	41.82
CriticalKV	26.03	31.50	48.39	41.73	27.10	18.84	25.30	24.10	25.99	71.00	86.30	43.16	2.66	87.23	56.69	53.20	41.83
AnDPro	25.78	**32.54**	**49.35**	42.20	26.94	18.63	25.33	24.01	26.29	71.00	86.30	**43.46**	2.68	86.98	57.18	53.77	42.03
LORE-KV /E (ours)	25.98	31.81	49.26	43.96	**27.17**	18.91	26.98	24.12	26.41	71.00	**86.60**	42.30	2.68	86.49	**57.83**	53.77	**42.20**
LORE-KV /D (ours)	**26.17**	31.10	49.22	**43.97**	27.08	18.82	26.99	**24.28**	26.38	**71.09**	86.54	42.33	2.61	86.51	57.71	**53.85**	42.17

et al., 2024), **RULER** (Hsieh et al., 2024), and **Needle-in-a-Haystack** (Nelson et al., 2024) using Mistral-7B-Instruct-v0.2 (Jiang et al., 2023) and Llama-3.1-8B-Instruct (Grattafiori et al., 2024). We compare against Full Cache and all KV-eviction baselines reported in Table 1. We sweep $B \in \{128, 256, 512, 1024\}$ with observation window $w = 32$ and chunk size 4; Table 6 lists the LongBench task types, metrics, input lengths and sample counts, and Table 5 the LORE-KV configuration used unchanged throughout. LongBench, Needle-in-a-Haystack, and RULER results are analyzed in Sections 3.1, 3.2, and 3.3, respectively; efficiency and mechanism analysis are reported in Sections 3.4 and 3.5, and the ablations in Table 10 follow the protocol of Appendix C. Full implementation details and LORE-KV hyperparameters are provided in Appendix D.

The two reliability variants. Every table reports both variants of Eq. equation 4 under an otherwise identical pipeline: LORE-KV /E, at $(\beta, \gamma, \lambda_D) = (1, 1.3, 0)$, whose numbers are complete, and LORE-KV /D, at $(1, 1, 0.3)$, whose end-to-end runs are in progress and are marked –. The two share particle generation, the perturbation score, budget allocation and selection, so any difference between the rows comes from the reliability coefficients alone—but note that they differ in *both* γ and λ_D (Proposition 1), so the pair is not a single-factor contrast; the single-factor cells $(1, 0, 0)$ and $(1, 0, \lambda_D)$ are reported as ablations in Table 10, and the two variants themselves are summarized in Appendix F. Their score-level comparison on identical captured particles is already available and is reported in Appendix E.5.

3.1 Evaluation on LongBench

Table 1 reports results on LongBench with Mistral-7B-Instruct-v0.2; the corresponding Llama-3.1-8B-Instruct results are reported in Table 7. LORE-KV /E achieves the best average performance at every cache budget. The gain is largest under aggressive compression: at $B = 128$ it reaches 40.28 versus 38.96 for the strongest baseline, AnDPro, while the gap narrows to 42.21 versus 42.03 at $B = 1024$. Both tables also report **LAQ++(8)**, a compute-matched extension of Lookahead Q-Cache (Wang et al., 2025b) that scores tokens from 8 local observation-window queries together with 8 forward Q-cache entries, matching LORE-KV /E's lookahead length T=8; results are avail-

Table 2: Baseline cache budgets required to match LORE-KV /E on LongBench with Mistral-7B-Instruct-v0.2, with corresponding cache savings.

LORE-KV /E budget	128	256	512	1024
Equivalent Ada-KV budget	∼332	∼745	>1024	>1024
Equivalent CriticalKV budget	∼307	∼709	>1024	>1024
Equivalent AnDPro budget	**∼208**	**∼414**	**∼743**	>1024
Budget saving vs. best baseline (%)	38.5	38.1	31.1	> 0

able at $B \in \{128, 256, 512\}$. Even against this equal-compute baseline, LORE-KV /E remains higher on average at every reported budget (e.g., 40.28 vs. 39.57 on Mistral and 46.11 vs. 38.70 on Llama at B=128, narrowing to 41.86 vs. 41.70 and 48.06 vs. 40.39 at B=512), though LAQ++(8) occasionally edges ahead on individual Mistral subtasks at B=128. Appendix E.5 isolates the reliability term itself at score level. We do not yet attribute this margin between the ensemble estimator and the choice of reliability channel: every $\lambda_D > 0$ cell is still –, and the two reported configurations are $(\beta, \gamma, \lambda_D) = (1, 1.3, 0)$ and $(1, 1, 0.3)$, which differ in *two* coefficients rather than one (Proposition 1), so they do not form a single-factor comparison.

We additionally measure how much cache a baseline requires to match the quality obtained by LORE-KV /E, reported in Table 2. At $B = 128$, the strongest baseline requires approximately 208 retained tokens to match LORE-KV /E, corresponding to a 38.5% cache saving; the saving remains 31.1% at $B = 512$. This provides an operational interpretation of the accuracy gains: under tight budgets, improved token selection translates directly into substantially lower cache requirements. Figure 10 shows the same trend as a continuous curve in behavioral fidelity, sustained across the whole budget sweep rather than at four discrete points.

3.2 Evaluation on Needle-in-a-Haystack

Needle-in-a-Haystack tests whether eviction preserves evidence needed by future decoding queries. We place the target at varying depths in 4K–32K contexts and retain $B = 1024$ tokens with Llama-3.1-8B-Instruct. As shown in the depth×length grid of Figure 3, retrieval is near saturation, and the methods mainly differ at extreme depths, where prompt-end queries are least representative of the evidence later required. LORE-KV /E achieves the highest exact-match average (0.977 vs. 0.955 for CriticalKV and 0.932 for AnDPro), with the same ordering under word recall. The gain therefore comes from preventing a small number of complete retrieval failures in precisely the cases where the prompt-side query reference is weakest. The corresponding LORE-KV /D run is in progress (–); because the two variants differ only in the scalar particle weights, they retain the same number of tokens and share the same latency at this budget, so any difference between them would be a retention-quality difference alone. Figure 8 gives the token-level counterpart, showing that LORE-KV /E retains answer-bearing tokens more often than the prompt-side criteria at the same budget.

3.3 Evaluation on RULER: Sweeping Context Length at a Fixed Task

RULER provides a controlled test of the query-reference hypothesis by instantiating the same 13 tasks at 4K, 8K, and 16K contexts while keeping the task and lookahead cost fixed. As shown in Table 3, LORE-KV /E achieves the best overall average in all eight model–budget settings and the best per-length result in 23 of 24 comparisons; the per-task breakdowns are Tables 8 and 9, which locate the gain in the aggregation-style tasks. The gains are largest where the prompt-side reference is expected to be weakest: under tight budgets, the margin over AnDPro is 5.05 points on Mistral and 3.52 on Llama at $B = 128$, but falls to about 0.4 points at $B = 1024$; with increasing context length, the gain also generally grows, e.g., from 3.58 points at 4K to 5.85 at 16K on Mistral with $B = 128$. Since the lookahead cost is unchanged across these settings, these trends support a retention-criterion effect rather than an extra-compute effect: future-query-aware scoring matters most when the cache is highly constrained and the fixed prompt-end window becomes less representative of later decoding queries. The only per-length exception is Llama at $B = 1024$, 16K, where CriticalKV scores 81.72 versus 79.98 for LORE-KV /E, consistent with the smaller differences between eviction criteria once the cache budget is relatively generous. The LORE-KV /D rows of Table 3 are –; since

Table 3: RULER accuracy averaged over 13 tasks at 4K, 8K, and 16K context lengths. **Avg.** is the mean across lengths.

Method	Mistral-7B-Instruct-v0.2				Llama-3.1-8B-Instruct			
	4K	8K	16K	Avg.	4K	8K	16K	Avg.
Budget = 128								
CriticalKV	43.83	38.75	36.08	39.55	50.72	48.85	45.06	48.21
AnDPro	54.85	50.06	45.20	50.04	60.06	58.42	52.80	57.09
LORE-KV /E (ours)	**58.40**	**55.77**	**51.05**	**55.07**	**64.27**	**63.43**	**56.05**	**61.25**
LORE-KV /D (ours)	58.31	55.74	51.00	55.02	64.20	63.32	55.98	61.17
Budget = 256								
CriticalKV	63.89	56.74	51.95	57.53	65.55	60.74	58.25	61.51
AnDPro	71.05	64.08	57.91	64.35	71.62	66.98	63.25	67.28
LORE-KV /E (ours)	**71.51**	**65.52**	**59.94**	**65.66**	**72.22**	**68.34**	**65.80**	**68.78**
LORE-KV /D (ours)	69.77	64.56	58.98	64.44	72.10	68.22	65.73	68.69
Budget = 512								
CriticalKV	79.26	67.75	63.65	70.22	79.25	69.02	67.38	71.88
AnDPro	83.42	72.06	67.12	74.20	82.71	72.58	70.28	75.19
LORE-KV /E (ours)	**83.69**	**72.82**	**68.28**	**74.93**	**83.12**	**73.46**	**71.60**	**76.06**
LORE-KV /D (ours)	81.67	72.18	64.98	72.94	83.05	73.35	71.53	75.98
Budget = 1024								
CriticalKV	88.78	83.05	80.25	84.03	88.35	83.26	**81.72**	84.44
AnDPro	90.97	85.31	82.06	86.11	93.38	85.94	79.37	86.23
LORE-KV /E (ours)	**91.11**	**85.74**	**82.75**	**86.53**	**93.60**	**86.40**	79.98	**86.66**
LORE-KV /D (ours)	89.88	84.51	81.53	85.31	93.50	86.29	79.87	86.55

RULER separates the criteria most sharply at B=128 and hardly at all at B=1024, that is where the two variants can be told apart, and it is the budget at which the LORE-KV /E-versus-LORE-KV /D comparison of Appendix F is run.

As a further compute-matched reference, **LAQ++(8)** (Wang et al., 2025b) reaches RULER averages of only 51.57, 62.81, and 69.83 on Llama-3.1-8B-Instruct at $B = 128, 256, 512$, respectively, versus 61.25, 68.78, and 76.06 for LORE-KV /E (Table 3)—a wider margin than on LongBench. Mistral-7B-v0.2 results, the $B = 1024$ setting, and the per-length and per-task breakdowns are not available for this baseline, so it is reported here in text rather than added to Table 3 or Tables 8–9.

3.4 Memory and Latency

At a fixed cache budget B, all methods have the same steady-state decoding memory and per-token cost. LORE-KV adds only a one-time lookahead overhead: the M particles are decoded as a single batched continuation of length T using the prefetched prompt KV cache. This cost is paid once before final generation and is independent of output length; the reliability term adds no decoding of its own (Appendix B.4), so both variants have the same cost profile. The full procedure is given in Algorithm 1, while Appendix B.4 details the computational accounting and compute-matched baseline control.

Because **LAQ++(8)** (Wang et al., 2025b) is evaluated with the same lookahead compute as LORE-KV /E (T=8), it isolates the effect of *how* that compute is used rather than how much is spent. LORE-KV /E still leads on average at every reported budget and dataset against this equal-compute baseline (LongBench: 40.28 vs. 39.57 on Mistral and 46.11 vs. 38.70 on Llama at B=128, narrowing but persisting to 41.86 vs. 41.70 and 48.06 vs. 40.39 at B=512; RULER on Llama: 61.25 vs. 51.57 at B=128 and 76.06 vs. 69.83 at B=512; Section 3.1 and Section 3.3). The advantage is therefore consistent and robust to matching test-time compute, indicating that it comes from the multi-trajectory, reliability-weighted estimator rather than from spending more test-time compute.

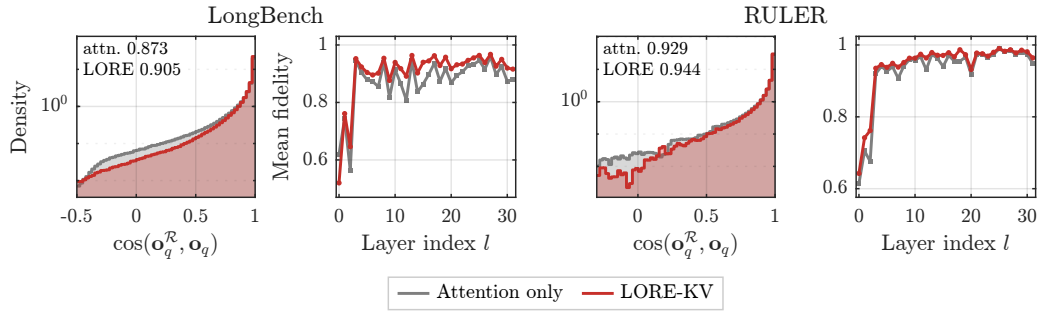


Figure 1: **Attention-output fidelity.** Cosine similarity between retained-cache and full-cache attention outputs on future queries, shown as distributions and layer-wise means. Mistral-7B-Instruct-v0.2, $B = 128$.

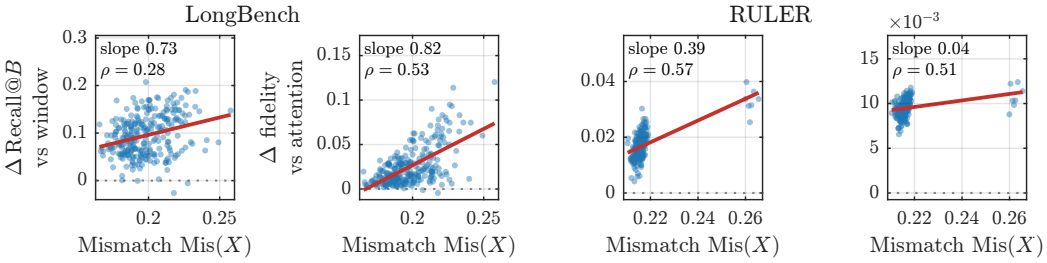


Figure 2: **Gain versus prompt-to-future query mismatch.** Per-example gains in oracle retention recall and output fidelity as mismatch increases. Mistral-7B-Instruct-v0.2, $B = 128$, $M = 4$.

3.5 Mechanism Analysis: Locating the Gain

We evaluate three predictions: P1 (*operating point*), P2 (*saturation*), and P3 (*locality*); Appendix D.1 restates them in full and Appendix E.1 details the protocol used to measure them below. Sections 3.1–3.3 establish P1; here we test P2 and P3 by directly measuring the query-reference mismatch identified in Section 1. Let $\mathcal{Q}_{\text{future}}$ denote the queries that the compressed cache actually serves (Appendix A). Since these queries are unavailable at eviction time, we record them afterwards from full-cache generation and use them only as ground truth for analysis. All results use Mistral-7B-Instruct-v0.2 at $B = 128$; the full protocol is provided in Appendix E.1, with additional analyses in Appendix E. Figure 4 shows that response-side lookahead queries better approximate $\mathcal{Q}_{\text{future}}$ than the prompt-end window: on LongBench, coverage improves from 0.788 to 0.826 with one rollout and saturates at 0.828-0.829 with additional rollouts. This supports P2: the main gain comes from replacing input-side with response-side queries, as in Section 2.2, rather than simply increasing rollout compute. The same saturation appears in Table 10. Figure 1 further shows that the resulting cache better preserves the full-cache output targeted by Eq. equation 11 and Eq. equation 7, increasing mean cosine similarity from 0.873 to 0.905 on LongBench and from 0.929 to 0.944 on RULER. Figure 9 shows that this improvement extends across the network, supporting the projected deletion criterion of Section 2.4.

To test P3, we measure each example's prompt-to-future query mismatch, $\text{Mis}(X) = 1 - \text{Cov}(\mathcal{Q}_{\text{future}}; \{\mathcal{Q}_{\mathcal{W}}^{l,h}\})$, and relate it to the LORE-KV /E gain in Figure 2. Larger mismatch consistently corresponds to larger gains: Spearman correlations are $\rho = 0.28$ and 0.57 for oracle retention recall, and $\rho = 0.53$ and 0.51 for output fidelity on LongBench and RULER, respectively. Together with the saturation in Figure 4 (P2) and the budget- and length-dependent results in Section 3.3 (P1), this supports the predicted mechanism: gains arise from correcting the query reference rather than merely adding rollout compute. Figure 4 additionally separates the two reliability variants: the entropy- and disagreement-weighted estimators are drawn as separate curves over the same nested particle prefixes, so the vertical distance between them is the entire effect of the penalty, and it is small next to the step from prompt-side to response-side queries. Additional checks are reported in

Appendix E, including Figures 5, 6, 7, 8, 9, and 10, with reliability weighting defined in Eq. equation 4.

4 Related Work

4.1 Prompt-side KV eviction

Most prior methods estimate token importance from prefill or prompt-side statistics. StreamingLLM preserves attention sinks and recent tokens (Xiao et al., 2024); H2O and Scissorhands use accumulated attention (Zhang et al., 2023; Liu et al., 2023); TOVA uses current-query attention (Oren et al., 2024); FastGen applies head-specific retention patterns (Ge et al., 2024); and SnapKV uses a prompt-end observation window (Li et al., 2024). PyramidKV and CAKE improve layer-wise budget allocation (Cai et al., 2024; Qin et al., 2025), while Ada-KV, HeadKV, DynamicKV, SAGE-KV, LAVa, and LU-KV refine allocation across heads, layers, or token groups (Feng et al., 2026; Fu et al., 2025; Zhou et al., 2024; Wang et al., 2025a; Shen et al., 2025; Tang et al., 2026). DefensiveKV further introduces robust aggregation against poorly estimated queries (Feng et al., 2025). These methods improve retention or budget allocation but still estimate importance using prompt-side queries rather than the future queries that consume the compressed cache.

4.2 Future-query estimation and output-aware scoring

Several methods improve the query reference. Lookahead Q-Cache uses queries from a single surrogate response (Wang et al., 2025b); Expected Attention analytically estimates future attention (Devoto et al., 2026); and DapQ constructs pseudo-queries from future positional information (Tian et al., 2025). LookaheadKV, KVP, globally calibrated retention methods, and IndexMem instead learn future token utility or compact memory representations (Ahn et al., 2026; Moschella et al., 2026; Bui et al., 2026; Yang et al., 2026). KVzip, FastKVzip, and ContrastKV target query-agnostic or multi-query robustness (Kim et al., 2026a;b; Chen et al., 2026b). A complementary line improves the importance score itself. CriticalKV incorporates value information (Guo et al., 2024); AnDPro uses projected attention-output information (Geng et al., 2026); and LaProx models projected values and inter-head effects (Mai & Kim, 2026). KeyDiff and L_2-norm methods use key geometry (Park et al., 2026; Devoto et al., 2024), while LagKV, GraphKV, R-KV, and MomentKV exploit temporal, graph, redundancy, or directional structure (Liang et al., 2025; Li et al., 2025b; Cai et al., 2025; Li et al., 2026). Recent analysis also shows that value-aware corrections depend strongly on the quality of the query reference (Zhang et al., 2026b). Future-aware methods improve the query reference but rely on single, analytic, or learned estimates; output-aware methods improve deletion scoring but remain prompt-side. Our reliability weighting connects this line to uncertainty estimation for free-form generation, where sequence likelihood, predictive entropy, and agreement across sampled continuations are the standard signals (Kuhn et al., 2023; Farquhar et al., 2024); the difference here is that agreement is measured directly between the sampled *query* trajectories, i.e. in the representation the eviction score consumes, rather than between the generated strings.

4.3 Orthogonal KV-cache compression

Other approaches reduce KV-cache cost along dimensions orthogonal to token eviction. Quantization methods such as KIVI, KVQuant, QAQ, MiKV, MiniKV, RotateKV, KVTuner, MixKVQ, PM-KVQ, and LeanKV reduce precision or combine sparsity with mixed precision (Liu et al., 2024b; Hooper et al., 2024; Dong et al., 2024; Yang et al., 2024; Sharma et al., 2024; Su et al., 2025; Li et al., 2025a; Zhang et al., 2026c; Liu et al., 2025; Zhang et al., 2025). GEAR, MiniCache, KVMerger, xKV, inter-layer methods, and transform coding exploit low-rank, cross-layer, mergeable, or coded representations (Kang et al., 2024; Liu et al., 2024a; Wang et al., 2024; Chang et al., 2026; Ma et al., 2026; Staniszewski & Łańcucki, 2026). KVServe targets communication-efficient serving (Liu et al., 2026), while DMS, ThinKV, RaaS, LazyEviction, R-KV, RocketKV, LongFlow, and CONF-KV address cache growth during long decoding (Łańcucki et al., 2026; Ramachandran et al., 2025; Hu et al., 2025; Zhang et al., 2026a; Cai et al., 2025; Behnam et al., 2025; Su et al., 2026; Li & Miao, 2026). These methods are complementary because they change cache precision, structure, serving, or decode-time maintenance rather than the prompt-time token-importance criterion. Aggressive eviction can also remove instructions or answer-bearing evidence (Yang et al.,

2024; Chen et al., 2026a; Garcia, 2026). Overall, prior methods either approximate future queries incompletely or improve deletion scoring while remaining prompt-side; the key gap is to combine multi-trajectory future-query estimation with future-aware deletion scoring.

5 CONCLUSION

LORE-KV reframes KV-cache eviction from identifying which prompt tokens were important to estimating which tokens will remain useful to future decoding queries, and it does so by *sampling* that future rather than predicting it: a small Monte-Carlo ensemble drawn from the frozen model replaces the prompt-side query reference. This addresses a central limitation of prior attention- and output-aware methods: their scores can be unreliable when computed from an unrepresentative prompt-side query reference. Making the ensemble an estimator also makes its weighting a design choice with a stated target. Reaching that target required separating a confound: because the particles are drawn from the model itself, a sampled token's log-probability already contains the entropy of the distribution it came from, so likelihood and uncertainty cannot be priced independently in the obvious parametrization. Centering the per-token score restores that independence and gives three coefficients—branch atypicality, context uncertainty, and ensemble-relative trajectory disagreement—which we compared under an otherwise identical pipeline. Our results show that the dominant effect is the substitution of response-side for prompt-side queries: lookahead is most useful under tight budgets and long contexts, provides little benefit when prompt-end queries already approximate future queries well, and the weighting rule is a second-order refinement on top of it. This motivates adaptively enabling lookahead only under high query mismatch, otherwise using prompt-only eviction, with future extensions to decode-time cache updates, adaptive particle counts driven by observed ensemble disagreement, and orthogonal compression methods.

AI USE STATEMENT

We used generative AI only to improve the grammar, clarity, and concision of author-written text, and coding assistants for routine implementation tasks such as plotting, table generation, and code refactoring. AI was not used to develop the research ideas, design the method or experiments, or generate and interpret results. All experimental results were obtained from actual runs or cited prior work. The authors reviewed and verified all AI-assisted content and take full responsibility for the final paper.

ETHICS STATEMENT

This work uses no human subjects, personally identifying data, or new data collection; experiments rely on publicly released LongBench, RULER, and Needle-in-a-Haystack benchmarks and the public Mistral-7B-Instruct-v0.2 and Llama-3.1-8B-Instruct checkpoints under their respective licenses. LORE-KV reduces KV-cache memory and bandwidth without adding new generative capabilities. However, aggressive eviction may remove instructions, boundaries, or evidence needed by later queries, potentially degrading instruction following or factual grounding; Appendix E.6 analyzes this behavior and Appendix G documents cases where the method is neutral or harmful. Deployments under tight cache budgets should therefore be validated on safety-relevant application prompts rather than average benchmark accuracy alone.

REPRODUCIBILITY STATEMENT

LORE-KV is training-free and modifies no model weights. The method is fully specified by Eqs. equation 1–equation 10 in Section 2, with pseudocode in Algorithm 1; all hyperparameters and default switches are listed in Table 5, and the two reliability variants are the points $(\beta, \gamma, \lambda_D) = (1, 1.3, 0)$ and $(1, 1, 0.3)$ of Eq. equation 4, so either is reproduced from the other by changing those coefficients. A run of the earlier (λ_H, η) parametrization is reproduced exactly by $(\beta, \gamma, \lambda_D) = (\eta, \eta(1 + \lambda_H), \eta\lambda_D)$ (Proposition 1). Appendix B.1 documents grouped-query-attention head mapping and scoring over $\mathcal{Q}^{l,h}_{\text{score}}$. Evaluation details include LongBench datasets, metrics, and sample counts in Table 6, RULER construction and scoring in Appendix D.3, and the budget grid, chunk size, and observation window in Section 3, which also distinguishes our runs

from values taken from prior work. All executed LongBench configurations are reported in Table 10, with RULER ablations in Table 11; Appendix E.1 specifies the mechanism-analysis capture procedure and plotted quantities, with sample sizes and lookahead budgets summarized in Table 12.

REFERENCES

Jinwoo Ahn, Ingyu Seong, Akhil Kedia, Junhan Kim, Hyemi Jang, Kangwook Lee, and Yongkweon Jeon. Lookaheadkv: Fast and accurate kv cache eviction by glimpsing into the future without generation. *arXiv preprint arXiv:2603.10899*, 2026.

Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. LongBench: A bilingual, multitask benchmark for long context understanding. pp. 3119–3137, August 2024. doi: 10.18653/v1/2024.acl-long.172. URL https://aclanthology.org/2024.acl-long.172/.

Payman Behnam, Yaosheng Fu, Ritchie Zhao, Po-An Tsai, Zhiding Yu, and Alexey Tumanov. Rocketkv: Accelerating long-context llm inference via two-stage kv cache compression. *arXiv preprint arXiv:2502.14051*, 2025.

Ngoc Bui, Hieu Trung Nguyen, Arman Cohan, and Rex Ying. Make each token count: Towards improving long-context performance with kv cache eviction. *arXiv preprint arXiv:2605.09649*, 2026.

Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Yucheng Li, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Junjie Hu, et al. Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling. *arXiv preprint arXiv:2406.02069*, 2024.

Zefan Cai, Wen Xiao, Hanshi Sun, Cheng Luo, Yikai Zhang, Ke Wan, Yucheng Li, Yeyang Zhou, Li-Wen Chang, Jiuxiang Gu, et al. R-kv: Redundancy-aware kv cache compression for training-free reasoning models acceleration. *arXiv e-prints*, pp. arXiv–2505, 2025.

Chi-Chih Chang, Wei-Cheng Lin, Chien-Yu Lin, Yash Akhauri, Hung-Yueh Chiang, Xilai Dai, Huiqiang Jiang, Yucheng Li, Kai-Chiang Wu, Luis Ceze, and Mohamed S. Abdelfattah. xKV: Cross-layer KV-cache compression via aligned singular vector extraction, 2026. URL https://openreview.net/forum?id=CSooB1sE2m.

Alex Chen, Renato Geh, Aditya Grover, Guy Van den Broeck, and Daniel Mingyi Israel. The pitfalls of kv cache compression. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 41530–41553, 2026a.

Xingchi Chen, Peiyuan Zong, Ziqiang Gao, Qing Li, Yong Jiang, Fa Zhu, and Hui Li. ContrastKV: Robust KV cache eviction via contrastive signal fusion for multi-query generalization. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (eds.), *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 9216–9229, San Diego, California, United States, July 2026b. Association for Computational Linguistics. ISBN 979-8-89176-390-6. doi: 10.18653/v1/2026.acl-long.417. URL https://aclanthology.org/2026.acl-long.417/.

Alessio Devoto, Yu Zhao, Simone Scardapane, and Pasquale Minervini. A simple and effective l_2 norm-based strategy for kv cache compression. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 18476–18499, 2024.

Alessio Devoto, Maximilian Jeblick, and Simon Jégou. Expected attention: KV cache compression by estimating attention from future queries distribution, 2026. URL https://openreview.net/forum?id=VmojW15eRc.

Shichen Dong, Wen Cheng, Jiayu Qin, and Wei Wang. Qaq: Quality adaptive quantization for llm kv cache. *arXiv preprint arXiv:2403.04643*, 2024.

Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630(8017):625–630, 2024.

Yuan Feng, Haoyu Guo, JunLin Lv, S Kevin Zhou, and Xike Xie. Taming the fragility of kv cache eviction in llm inference. *arXiv preprint arXiv:2510.13334*, 2025.

Yuan Feng, Junlin Lv, Yukun Cao, Xike Xie, and S Kevin Zhou. Ada-kv: Optimizing kv cache eviction by adaptive budget allocation for efficient llm inference. *Advances in Neural Information Processing Systems*, 38:113152–113188, 2026.

Yu Fu, Zefan Cai, Abedelkadir Asi, Wayne Xiong, Yue Dong, and Wen Xiao. Not all heads matter: A head-level KV cache compression method with integrated retrieval and reasoning. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=FJFVmeXusW.

Gabriel Garcia. Protection is (nearly) all you need: Structural protection dominates scoring in globally capped kv eviction. *arXiv preprint arXiv:2605.18053*, 2026.

Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. Model tells you what to discard: Adaptive kv cache compression for llms. In *International Conference on Learning Representations*, volume 2024, pp. 22975–22988, 2024.

Zijie Geng, Jie Wang, Ziqi Liu, Feng Ju, Yiming Li, Xing Li, Mingxuan Yuan, Jianye Hao, Defu Lian, Enhong Chen, et al. Accurate kv cache eviction via anchor direction projection for efficient llm inference. *Advances in Neural Information Processing Systems*, 38:10420–10456, 2026.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurelien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Roziere, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, Chloe Bi, Chris Marra, Chris McConnell, Christian Keller, Christophe Touret, Chunyang Wu, Corinne Wong, Cristian Canton Ferrer, Cyrus Nikolaidis, Damien Allonsius, Daniel Song, Danielle Pintz, Danny Livshits, Danny Wyatt, David Esiobu, Dhruv Choudhary, Dhruv Mahajan, Diego Garcia-Olano, Diego Perino, Dieuwke Hupkes, Egor Lakomkin, Ehab AlBadawy, Elina Lobanova, Emily Dinan, Eric Michael Smith, Filip Radenovic, Francisco Guzmán, Frank Zhang, Gabriel Synnaeve, Gabrielle Lee, Georgia Lewis Anderson, Govind Thattai, Graeme Nail, Gregoire Mialon, Guan Pang, Guillem Cucurell, Hailey Nguyen, Hannah Korevaar, Hu Xu, Hugo Touvron, Iliyan Zarov, Imanol Arrieta Ibarra, Isabel Kloumann, Ishan Misra, Ivan Evtimov, Jack Zhang, Jade Copet, Jaewon Lee, Jan Geffert, Jana Vranes, Jason Park, Jay Mahadeokar, Jeet Shah, Jelmer van der Linde, Jennifer Billock, Jenny Hong, Jenya Lee, Jeremy Fu, Jianfeng Chi, Jianyu Huang, Jiawen Liu, Jie Wang, Jiecao Yu, Joanna Bitton, Joe Spisak, Jongsoo Park, Joseph Rocca, Joshua Johnstun, Joshua Saxe, Junteng Jia, Kalyan Vasuden Alwala, Karthik Prasad, Kartikeya Upasani, Kate Plawiak, Ke Li, Kenneth Heafield, Kevin Stone, Khalid El-Arini, Krithika Iyer, Kshitiz Malik, Kuenley Chiu, Kunal Bhalla, Kushal Lakhotia, Lauren Rantala-Yeary, Laurens van der Maaten, Lawrence Chen, Liang Tan, Liz Jenkins, Louis Martin, Lovish Madaan, Lubo Malo, Lukas Blecher, Lukas Landzaat, Luke de Oliveira, Madeline Muzzi, Mahesh Pasupuleti, Mannat Singh, Manohar Paluri, Marcin Kardas, Maria Tsimpoukelli, Mathew Oldham, Mathieu Rita, Maya Pavlova, Melanie Kambadur, Mike Lewis, Min Si, Mitesh Kumar Singh, Mona Hassan, Naman Goyal, Narjes Torabi, Nikolay Bashlykov, Nikolay Bogoychev, Niladri Chatterji, Ning Zhang, Olivier Duchenne, Onur Çelebi, Patrick Alrassy, Pengchuan Zhang, Pengwei Li, Petar Vasic, Peter Weng, Prajjwal Bhargava, Pratik Dubal, Praveen Krishnan, Punit Singh Koura, Puxin Xu, Qing He, Qingxiao Dong, Ragavan Srinivasan, Raj Ganapathy, Ramon Calderer, Ricardo Silveira Cabral, Robert Stojnic, Roberta Raileanu, Rohan Maheswari, Rohit Girdhar, Rohit Patel, Romain Sauvestre, Ronnie Polidoro, Roshan Sumbaly, Ross Taylor, Ruan Silva, Rui Hou, Rui Wang, Saghar Hosseini, Sahana Chennabasappa, Sanjay Singh, Sean Bell, Seohyun Sonia Kim, Sergey Edunov, Shaoliang Nie, Sharan Narang, Sharath Raparthy, Sheng Shen, Shengye Wan, Shruti Bhosale, Shun Zhang, Simon Vandenhende, Soumya Batra, Spencer Whitman, Sten Sootla, Stephane Collot, Suchin Gururangan, Sydney Borodinsky, Tamar Herman, Tara Fowler, Tarek Sheasha, Thomas Georgiou, Thomas Scialom, Tobias Speckbacher, Todor Mihaylov, Tong Xiao, Ujjwal Karn, Vedanuj Goswami, Vibhor Gupta, Vignesh Ramanathan, Viktor Kerkez, Vincent Gonguet, Virginie Do, Vish Vogeti, Vítor Albiero, Vladan Petrovic, Weiwei Chu, Wenhan Xiong, Wenyin Fu, Whitney Meers, Xavier Martinet, Xiaodong Wang, Xiaofang

Wang, Xiaoqing Ellen Tan, Xide Xia, Xinfeng Xie, Xuchao Jia, Xuewei Wang, Yaelle Goldschlag, Yashesh Gaur, Yasmine Babaei, Yi Wen, Yiwen Song, Yuchen Zhang, Yue Li, Yuning Mao, Zacharie Delpierre Coudert, Zheng Yan, Zhengxing Chen, Zoe Papakipos, Aaditya Singh, Aayushi Srivastava, Abha Jain, Adam Kelsey, Adam Shajnfeld, Adithya Gangidi, Adolfo Victoria, Ahuva Goldstand, Ajay Menon, Ajay Sharma, Alex Boesenberg, Alexei Baevski, Allie Feinstein, Amanda Kallet, Amit Sangani, Amos Teo, Anam Yunus, Andrei Lupu, Andres Alvarado, Andrew Caples, Andrew Gu, Andrew Ho, Andrew Poulton, Andrew Ryan, Ankit Ramchandani, Annie Dong, Annie Franco, Anuj Goyal, Aparajita Saraf, Arkabandhu Chowdhury, Ashley Gabriel, Ashwin Bharambe, Assaf Eisenman, Azadeh Yazdan, Beau James, Ben Maurer, Benjamin Leonhardi, Bernie Huang, Beth Loyd, Beto De Paola, Bhargavi Paranjape, Bing Liu, Bo Wu, Boyu Ni, Braden Hancock, Bram Wasti, Brandon Spence, Brani Stojkovic, Brian Gamido, Britt Montalvo, Carl Parker, Carly Burton, Catalina Mejia, Ce Liu, Changhan Wang, Changkyu Kim, Chao Zhou, Chester Hu, Ching-Hsiang Chu, Chris Cai, Chris Tindal, Christoph Feichtenhofer, Cynthia Gao, Damon Civin, Dana Beaty, Daniel Kreymer, Daniel Li, David Adkins, David Xu, Davide Testuggine, Delia David, Devi Parikh, Diana Liskovich, Didem Foss, Dingkang Wang, Duc Le, Dustin Holland, Edward Dowling, Eissa Jamil, Elaine Montgomery, Eleonora Presani, Emily Hahn, Emily Wood, Eric-Tuan Le, Erik Brinkman, Esteban Arcaute, Evan Dunbar, Evan Smothers, Fei Sun, Felix Kreuk, Feng Tian, Filippos Kokkinos, Firat Ozgenel, Francesco Caggioni, Frank Kanayet, Frank Seide, Gabriela Medina Florez, Gabriella Schwarz, Gada Badeer, Georgia Swee, Gil Halpern, Grant Herman, Grigory Sizov, Guangyi, Zhang, Guna Lakshminarayanan, Hakan Inan, Hamid Shojanazeri, Han Zou, Hannah Wang, Hanwen Zha, Haroun Habeeb, Harrison Rudolph, Helen Suk, Henry Aspegren, Hunter Goldman, Hongyuan Zhan, Ibrahim Damlaj, Igor Molybog, Igor Tufanov, Ilias Leontiadis, Irina-Elena Veliche, Itai Gat, Jake Weissman, James Geboski, James Kohli, Janice Lam, Japhet Asher, Jean-Baptiste Gaya, Jeff Marcus, Jeff Tang, Jennifer Chan, Jenny Zhen, Jeremy Reizenstein, Jeremy Teboul, Jessica Zhong, Jian Jin, Jingyi Yang, Joe Cummings, Jon Carvill, Jon Shepard, Jonathan McPhie, Jonathan Torres, Josh Ginsburg, Junjie Wang, Kai Wu, Kam Hou U, Karan Saxena, Kartikay Khandelwal, Katayoun Zand, Kathy Matosich, Kaushik Veeraraghavan, Kelly Michelena, Keqian Li, Kiran Jagadeesh, Kun Huang, Kunal Chawla, Kyle Huang, Lailin Chen, Lakshya Garg, Lavender A, Leandro Silva, Lee Bell, Lei Zhang, Liangpeng Guo, Licheng Yu, Liron Moshkovich, Luca Wehrstedt, Madian Khabsa, Manav Avalani, Manish Bhatt, Martynas Mankus, Matan Hasson, Matthew Lennie, Matthias Reso, Maxim Groshev, Maxim Naumov, Maya Lathi, Meghan Keneally, Miao Liu, Michael L. Seltzer, Michal Valko, Michelle Restrepo, Mihir Patel, Mik Vyatskov, Mikayel Samvelyan, Mike Clark, Mike Macey, Mike Wang, Miquel Jubert Hermoso, Mo Metanat, Mohammad Rastegari, Munish Bansal, Nandhini Santhanam, Natascha Parks, Natasha White, Navyata Bawa, Nayan Singhal, Nick Egebo, Nicolas Usunier, Nikhil Mehta, Nikolay Pavlovich Laptev, Ning Dong, Norman Cheng, Oleg Chernoguz, Olivia Hart, Omkar Salpekar, Ozlem Kalinli, Parkin Kent, Parth Parekh, Paul Saab, Pavan Balaji, Pedro Rittner, Philip Bontrager, Pierre Roux, Piotr Dollar, Polina Zvyagina, Prashant Ratanchandani, Pritish Yuvraj, Qian Liang, Rachad Alao, Rachel Rodriguez, Rafi Ayub, Raghotham Murthy, Raghu Nayani, Rahul Mitra, Rangaprabhu Parthasarathy, Raymond Li, Rebekkah Hogan, Robin Battey, Rocky Wang, Russ Howes, Ruty Rinott, Sachin Mehta, Sachin Siby, Sai Jayesh Bondu, Samyak Datta, Sara Chugh, Sara Hunt, Sargun Dhillon, Sasha Sidorov, Satadru Pan, Saurabh Mahajan, Saurabh Verma, Seiji Yamamoto, Sharadh Ramaswamy, Shaun Lindsay, Shaun Lindsay, Sheng Feng, Shenghao Lin, Shengxin Cindy Zha, Shishir Patil, Shiva Shankar, Shuqiang Zhang, Shuqiang Zhang, Sinong Wang, Sneha Agarwal, Soji Sajuyigbe, Soumith Chintala, Stephanie Max, Stephen Chen, Steve Kehoe, Steve Satterfield, Sudarshan Govindaprasad, Sumit Gupta, Summer Deng, Sungmin Cho, Sunny Virk, Suraj Subramanian, Sy Choudhury, Sydney Goldman, Tal Remez, Tamar Glaser, Tamara Best, Thilo Koehler, Thomas Robinson, Tianhe Li, Tianjun Zhang, Tim Matthews, Timothy Chou, Tzook Shaked, Varun Vontimitta, Victoria Ajayi, Victoria Montanez, Vijai Mohan, Vinay Satish Kumar, Vishal Mangla, Vlad Ionescu, Vlad Poenaru, Vlad Tiberiu Mihailescu, Vladimir Ivanov, Wei Li, Wenchen Wang, Wenwen Jiang, Wes Bouaziz, Will Constable, Xiaocheng Tang, Xiaojian Wu, Xiaolan Wang, Xilun Wu, Xinbo Gao, Yaniv Kleinman, Yanjun Chen, Ye Hu, Ye Jia, Ye Qi, Yenda Li, Yilin Zhang, Ying Zhang, Yossi Adi, Youngjin Nam, Yu, Wang, Yu Zhao, Yuchen Hao, Yundi Qian, Yunlu Li, Yuzi He, Zach Rait, Zachary DeVito, Zef Rosnbrick, Zhaoduo Wen, Zhenyu Yang, Zhiwei Zhao, and Zhiyu Ma. The llama 3 herd of models. 2024. URL https://arxiv.org/abs/2407.21783.

Zhiyu Guo, Hidetaka Kamigaito, and Taro Watanabe. Attention score is not all you need for token importance indicator in kv cache reduction: Value also matters. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 21158–21166, 2024.

Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, Yakun S Shao, Kurt Keutzer, and Amir Gholami. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *Advances in Neural Information Processing Systems*, 37:1270–1303, 2024.

Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. Ruler: What's the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, 2024.

Junhao Hu, Wenrui Huang, Weidong Wang, Zhenwen Li, Tiancheng Hu, Zhixia Liu, Xusheng Chen, Tao Xie, and Yizhou Shan. Raas: Reasoning-aware attention sparsity for efficient llm reasoning. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 2577–2590, 2025.

Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7b. 2023. URL https://arxiv.org/abs/2310.06825.

Hao Kang, Qingru Zhang, Souvik Kundu, Geonhwa Jeong, Zaoxing Liu, Tushar Krishna, and Tuo Zhao. Gear: An efficient kv cache compression recipe for near-lossless generative inference of llm. *arXiv preprint arXiv:2403.05527*, 2024.

Jang-Hyun Kim, Dongyoon Han, and Sangdoo Yun. Fast kvzip: Efficient and accurate llm inference with gated kv eviction. *arXiv preprint arXiv:2601.17668*, 2026a.

Jang-Hyun Kim, Jinuk Kim, Sangwoo Kwon, Jae W Lee, Sangdoo Yun, and Hyun Oh Song. Kvzip: Query-agnostic kv cache compression with context reconstruction. *Advances in Neural Information Processing Systems*, 38:167563–167591, 2026b.

Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. 2023.

Adrian Łańcucki, Konrad Staniszewski, Piotr Nawrot, and Edoardo Maria Ponti. Inference-time hyper-scaling with kv cache compression. *Advances in Neural Information Processing Systems*, 38:9365–9397, 2026.

Xing Li, Zeyu Xing, Yiming Li, Linping Qu, Hui-Ling Zhen, Wulong Liu, Yiwu Yao, Sinno Jialin Pan, and Mingxuan Yuan. Kvtuner: Sensitivity-aware layer-wise mixed-precision kv cache quantization for efficient and nearly lossless llm inference. *arXiv preprint arXiv:2502.04420*, 2025a.

Xuelin Li, Xiangqi Jin, and Linfeng Zhang. Graphkv: Breaking the static selection paradigm with graph-based kv cache eviction. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 21910–21920, 2025b.

Yu Li, Binxu Li, and Tian Lan. Momentkv: Closing the directional gap in kv cache eviction for long-context inference. *arXiv preprint arXiv:2606.01563*, 2026.

Yubo Li and Yidi Miao. Conf-kv: Confidence-aware kv cache eviction with mixed-precision storage for long-horizon llm. *arXiv preprint arXiv:2605.24786*, 2026.

Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. Snapkv: Llm knows what you are looking for before generation. *Advances in Neural Information Processing Systems*, 37:22947–22970, 2024.

Manlai Liang, JiaMing Zhang, Xiong Li, and Jinlong Li. Lagkv: Lag-relative information of the kv cache tells which tokens are important. *arXiv preprint arXiv:2504.04704*, 2025.

Akide Liu, Jing Liu, Zizheng Pan, Yefei He, Gholamreza Haffari, and Bohan Zhuang. Minicache: Kv cache compression in depth dimension for large language models, 2024a. URL https://arxiv.org/abs/2405.14366.

Tengxuan Liu, Shiyao Li, Jiayi Yang, Tianchen Zhao, Feng Zhou, Xiaohui Song, Guohao Dai, Shengen Yan, Huazhong Yang, and Yu Wang. Pm-kvq: Progressive mixed-precision kv cache quantization for long-cot llms. *arXiv preprint arXiv:2505.18610*, 2025.

Zedong Liu, Xinyang Ma, Dejun Luo, Hairui Zhao, Bing Lu, Wenjing Huang, Yida Gu, Xingchen Liu, Zheng Wei, Jinyang Liu, et al. Kvserve: Service-aware kv cache compression for communication-efficient disaggregated llm serving. *arXiv preprint arXiv:2605.13734*, 2026.

Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. Scissorhands: Exploiting the persistence of importance hypothesis for llm kv cache compression at test time. *Advances in Neural Information Processing Systems*, 36:52342–52364, 2023.

Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*, 2024b.

Da Ma, Lu Chen, Situo Zhang, Yuxun Miao, Su Zhu, Zhi Chen, Hongshen Xu, Hanqi Li, Shuai Fan, Lei Pan, and Kai Yu. Compressing kv cache for long-context llm inference with inter-layer attention similarity. In *ICASSP 2026 - 2026 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 16407–16411, 2026. doi: 10.1109/ICASSP55912.2026.11464826.

Tho Mai and Joo-Young Kim. Reformulating kv cache eviction problem for long-context llm inference. *arXiv preprint arXiv:2605.07234*, 2026.

Luca Moschella, Laura Manduchi, and Ozan Sener. Learning to evict from key-value cache. *arXiv preprint arXiv:2602.10238*, 2026.

Elliot Nelson, Georgios Kollias, Payel Das, Subhajit Chaudhury, and Soham Dan. Needle in the haystack for memory based large language models, 2024. URL https://arxiv.org/abs/2407.01437.

Matanel Oren, Michael Hassid, Nir Yarden, Yossi Adi, and Roy Schwartz. Transformers are multi-state rnns. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 18724–18741, 2024.

Junyoung Park, Dalton Jones, Matthew Morse, Raghavv Goel, Mingu Lee, and Christopher Lott. Keydiff: Key similarity-based kv cache eviction for long-context llm inference in resource-constrained environments. *Advances in Neural Information Processing Systems*, 38:5983–6019, 2026.

Ziran Qin, Yuchen Cao, Mingbao Lin, Wen Hu, Shixuan Fan, Ke Cheng, Weiyao Lin, and Jianguo Li. CAKE: Cascading and adaptive KV cache eviction with layer preferences. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=EQgEMAD4kv.

Akshat Ramachandran, Marina Neseem, Charbel Sakr, Rangharajan Venkatesan, Brucek Khailany, and Tushar Krishna. Thinkv: Thought-adaptive kv cache compression for efficient reasoning models. *arXiv preprint arXiv:2510.01290*, 2025.

Akshat Sharma, Hangliang Ding, Jianping Li, Neel Dani, and Minjia Zhang. Minikv: Pushing the limits of llm inference via 2-bit layer-discriminative kv cache. *arXiv preprint arXiv:2411.18077*, 2024.

Yiqun Shen, Song Yuan, Zhengze Zhang, Xiaoliang Wang, Daxin Jiang, and Nguyen Cam-Tu. Lava: Layer-wise kv cache eviction with dynamic budget allocation, 2025. URL https://arxiv.org/abs/2509.09754.

Konrad Staniszewski and Adrian Łańcucki. KV cache transform coding for compact storage in LLM inference. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=aNVKROYpLB.

Yi Su, Zhenxu Tian, Dan Qiao, Yuechi Zhou, Juntao Li, and Min Zhang. Longflow: Efficient kv cache compression for reasoning models. *arXiv preprint arXiv:2603.11504*, 2026.

Zunhai Su, Zhe Chen, Wang Shen, Hanyu Wei, Linge Li, Huangqi Yu, and Kehong Yuan. Rotatekv: Accurate and robust 2-bit kv cache quantization for llms via outlier-aware adaptive rotations. *arXiv preprint arXiv:2501.16383*, 2025.

Ziyao Tang, Pengkun Jiao, Xinhang Chen, Wei Liu, Shiyong Li, and Jingjing Chen. Predicting future utility: Global combinatorial optimization for task-agnostic kv cache eviction. *arXiv preprint arXiv:2602.08585*, 2026.

Zhenxu Tian, Yi Su, Juntao Li, and Min Zhang. Where matters more than what: Decoding-aligned KV cache compression via position-aware pseudo-queries, 2025. URL https://openreview.net/forum?id=ZpgyPxdxiL.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

Guangtao Wang, Shubhangi Upasani, Chen Wu, Darshan Gandhi, Jonathan Li, Changran Hu, Bo Li, and Urmish Thakker. Llms know what to drop: Self-attention guided kv cache eviction for efficient long-context inference. *arXiv preprint arXiv:2503.08879*, 2025a.

Yixuan Wang, Shiyu Ji, Yijun Liu, Yuzhuang Xu, Yang Xu, Qingfu Zhu, and Wanxiang Che. Lookahead q-cache: Achieving more consistent kv cache eviction via pseudo query. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 34146–34162, 2025b.

Zheng Wang, Boxiao Jin, Zhongzhi Yu, and Minjia Zhang. Model tells you where to merge: Adaptive kv cache merging for llms on long-context tasks, 2024. URL https://arxiv.org/abs/2407.08454.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations*, volume 2024, pp. 21875–21895, 2024.

June Yong Yang, Byeongwook Kim, Jeongin Bae, Beomseok Kwon, Gunho Park, Eunho Yang, Se Jung Kwon, and Dongsoo Lee. No token left behind: Reliable kv cache compression via importance-aware mixed precision quantization. *arXiv preprint arXiv:2402.18096*, 2024.

Xintong Yang, Hao Gu, Binxing Xu, Lujun Li, Bei Liu, Jiacheng Liu, Qiyuan Zhu, Sirui Han, and Yike Guo. Indexmem: Learned kv-cache eviction with latent memory for long-context llm inference. *arXiv preprint arXiv:2605.25475*, 2026.

Haoyue Zhang, Hualei Zhang, Xiaosong Ma, Jie Zhang, and Song Guo. Lazyeviction: Lagged kv eviction with attention pattern observation for efficient long reasoning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 36335–36352, 2026a.

Ruijie Zhang, Haozhe Liang, Da Chang, Li Hu, Fanqi Kong, Huaxiao Yin, and Yu Li. When does value-aware kv eviction help? a fixed-contract diagnostic for non-monotone cache compression, 2026b. URL https://arxiv.org/abs/2605.08234.

Tao Zhang, Ziqian Zeng, Hao Peng, Huiping Zhuang, and Cen Chen. Mixkvq: Query-aware mixed-precision kv cache quantization for long-context reasoning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 7189–7204, 2026c.

Yanqi Zhang, Yuwei Hu, Runyuan Zhao, John C. S. Lui, and Haibo Chen. Diffkv: Differentiated memory management for large language models with parallel kv compaction. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, SOSP '25, pp. 431–445, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400718700. doi: 10.1145/3731569.3764810. URL https://doi.org/10.1145/3731569.3764810.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 36:34661–34710, 2023.

Xiabin Zhou, Wenbin Wang, Minyan Zeng, Jiaxian Guo, Xuebo Liu, Li Shen, Min Zhang, and Liang Ding. Dynamickv: Task-aware adaptive kv cache compression for long context llms. *arXiv preprint arXiv:2412.14838*, 2024.

A Background: The KV Cache and the Eviction Problem

KV caching and memory cost. A decoder-only Transformer (Vaswani et al., 2017) generates tokens autoregressively, projecting each hidden state into a query $\mathbf{q}_t$, key $\mathbf{k}_t$, and value $\mathbf{v}_t$, with attention weights determined by $\mathbf{q}_t^\top \mathbf{k}_j/\sqrt{d}$. Since past keys and values remain fixed during inference, they are stored in the *KV cache*, enabling a parallel *prefill* over the prompt followed by token-by-token decoding. For L layers, H KV heads, head dimension d, sequence length n, batch size b, and p bytes per element, the cache requires $\mathrm{Mem}_{\mathrm{KV}} = 2bLHdnp$ bytes, where the factor 2 accounts for keys and values.Thus, memory grows linearly with context length even at batch size one. For example, consider a 70B-class model with $L = 80$ layers, $H = 8$ KV heads, $d = 128$, a 128K-token context ($n = 131{,}072$), $b = 1$, and $p = 2$ bytes (BF16). Its KV cache alone requires $\mathrm{Mem}_{\mathrm{KV}} \approx 42.9$ GB. For comparison, a 70B-parameter model stored in BF16 requires roughly 140 GB for its weights, so a single 128K context adds KV-cache memory equal to about 31% of the model's weight memory.

The eviction problem. Given a cache budget $B \ll n$, eviction retains index sets $\mathcal{R}^{l,h} \subseteq \{1, \ldots, n\}$ with $|\mathcal{R}^{l,h}| \leq B_{l,h}$ and removes the remaining KV pairs. Ideally, the retained cache should preserve the behavior of the full-cache model during future decoding:

$$\min_{\{\mathcal{R}^{l,h}\}} \mathbb{E}_{\mathbf{q}\sim\mathcal{Q}_{\mathrm{future}}} \left[\mathrm{dist}(\mathrm{output}(\mathbf{q}; \text{full cache}), \mathrm{output}(\mathbf{q}; \text{retained cache}))\right]. \tag{11}$$

Future decoding queries $\mathcal{Q}_{\mathrm{future}}$ are unknown at eviction time, so methods must estimate which cached tokens will matter. Prior work uses prompt-side or predicted queries with attention- or output-based scores, whereas LORE-KV uses confidence-weighted lookahead queries and ranks tokens by their leave-one-out output impact. Section 3.5 evaluates this approximation against the actual future queries observed after generation.

B Method Details

This appendix discusses the detailed design, pseudocode, cost accounting, and implementation of the methodology in Section 2.

B.1 Implementation: Protected Set, Grouped-Query Attention, Fused Kernels

Protected positions. The optional set $\mathcal{B}$ holds the positions of prompt-structure markers: chat-template delimiters (`<|system|>`, `<|user|>`, `<|assistant|>`, `[INST]`, `[/INST]`), tool-call markers and document separators. These are read off the tokenized prompt and always retained; if the template exposes none, $\mathcal{B} = \emptyset$. Following prior work (Xiao et al., 2024; Li et al., 2024), this keeps instructions and prompt boundaries from being removed under aggressive eviction.

Grouped-query attention. Under GQA/MQA, multiple query heads share each KV head. We therefore score each cached pair $(\mathbf{k}_i^{l,h}, \mathbf{v}_i^{l,h})$ using all query heads in its group $\mathcal{G}(h)$ and aggregate their deletion effects from Eq. equation 6. Throughout, H denotes the number of KV heads, and cache budgets are assigned at the KV-head level.

Fused attention kernels. Kernels such as FlashAttention never materialize the $n\times n$ attention matrix a perturbation score needs. We avoid kernel surgery by computing scores only for the scoring-query set $\mathcal{Q}_{\mathrm{score}}^{l,h}$ of Eq. equation 2, whose size $|\mathcal{Q}_{\mathrm{score}}^{l,h}| = w + MT$ is far below n. For those few queries we form the attention weights against the cached keys explicitly, leaving the fused kernel untouched for the rest of prefill and for final decoding. Eviction scoring is thus a small side computation that never requires full-matrix attention on the main inference path.

Table 4: Average inference time per sample (seconds) across datasets, backbones, and cache budgets. Lower is better.

Backbone	Dataset	Method	128	256	512	1024
Mistral	LongBench	AndPro	7.8799	7.4885	7.4355	7.5338
		CriticalKV	6.6867	6.3991	6.3982	6.5276
		LORE-KV /E	17.5342	19.5496	19.6767	17.7930
	RULER	AndPro	6.2360	4.5775	5.9707	3.1746
		CriticalKV	6.1907	4.1849	4.9899	2.4039
		LORE-KV /E	13.6877	11.1720	14.5537	7.2315
	NIAH	AndPro	2.9842	2.8559	2.8779	2.9169
		CriticalKV	2.3514	2.1441	2.1109	2.1961
		LORE-KV /E	7.3873	7.4716	7.4072	7.3407
LLaMA	LongBench	AndPro	8.3347	8.2447	8.1548	8.0648
		CriticalKV	7.0726	7.0443	7.0160	6.9877
		LORE-KV /E	12.1815	14.4038	15.8882	13.1342
	RULER	AndPro	4.3309	4.0226	3.7143	3.4060
		CriticalKV	4.2994	3.6776	3.1042	2.5791
		LORE-KV /E	6.6410	6.8587	6.3249	5.4202
	NIAH	AndPro	2.6046	2.5137	2.5269	2.6172
		CriticalKV	1.9267	1.9222	1.9450	1.8561
		LORE-KV /E	7.0298	6.9553	7.0076	6.9816

B.2 Relation to Prior Surrogate-Query and Output-Aware Methods

LORE-KV inherits two components and adds one. Response-side surrogate queries for estimating future cache usage come from Lookahead Q-Cache (Wang et al., 2025b; Ahn et al., 2026), and output-aware deletion scoring beyond attention alone comes from AnDPro and its relatives (Guo et al., 2024; Geng et al., 2026; Mai & Kim, 2026). What is added is that the surrogate is a *Monte-Carlo ensemble* rather than a single continuation, together with the reliability weighting that turns the ensemble into an estimator with a stated target distribution (Section 2.3). Concretely, Lookahead Q-Cache is the $M{=}1$ special case of the query construction and AnDPro is the $\lambda_L{=}0$ special case of the score, so both are reachable from Eqs. equation 2 and equation 8 by switch settings, and both are priced in Tables 10 and 10. We weight the trajectories using model likelihood together with either predictive entropy (Kuhn et al., 2023; Farquhar et al., 2024), which is LORE-KV /E, or ensemble disagreement measured in query space, which is LORE-KV /D. The two inherited components are common to both variants, so the $M{=}1$ and $\lambda_L{=}0$ special cases are shared: at $M{=}1$ the softmax of Eq. equation 5 normalizes a single scalar away and $\hat{D}_m$ is identically zero, so LORE-KV /E and LORE-KV /D coincide exactly with Lookahead Q-Cache-style scoring, and the variants can only differ at $M \geq 3$. Figure 7 measures both signals against the same per-particle oracle utility and ranks them against each other on identical particles, while Figure 6 shows that either confidence is only modestly correlated with that utility, motivating soft weighting rather than single-trajectory selection.

B.3 Reliability Weighting: Derivations, Calibration, and Edge Cases

This section collects the derivations and implementation details omitted from Section 2.3.

Centering and reparameterization. The centered score of Eq. equation 3 removes the context-entropy baseline from the likelihood channel in conditional expectation. For $y_u \sim q(\cdot \mid h_u)$,

$$\mathbb{E}[c_u \mid h_u] = \sum_y q(y \mid h_u) \log q(y \mid h_u) + H_u = -H_u + H_u = 0, \tag{12}$$

so $\mathbb{E}[\bar{c}_m] = 0$. Moreover, $S_t = \sum_{u \leq t} c_u$ is a martingale with respect to the natural filtration. This centering should not be read as statistical independence between $\bar{c}_m$ and $\bar{H}_m$; it makes branch

Algorithm 1 LORE-KV eviction procedure.

Require: frozen LLM f_θ; prompt $X = (x_1, \dots, x_n)$; window w; particles M, length T; layer budgets $\{B_l\}_{l=1}^{L}$; core constants $\epsilon, \beta, \gamma, \lambda_D, \lambda_L$; plug-in switches $\alpha, \kappa, \tau, \lambda_k, \lambda_v$ ▷ all switches $= 0$ gives the core method; $(\beta, \gamma, \lambda_D) = (1, 1.3, 0)$ is LORE-KV /E and $(1, 1, 0.3)$ is LORE-KV /D

Ensure: compressed KV cache $(\mathbf{K}^\star, \mathbf{V}^\star)$

$\big(\{\mathbf{K}^{l,h}, \mathbf{V}^{l,h}\}, \{\mathcal{Q}_{\mathcal{W}}^{l,h}\}\big) \leftarrow \mathrm{Prefill}(f_\theta, X)$ ▷ build the KV cache and collect prompt-end queries

$\mathcal{P} \leftarrow \{1\} \cup \mathcal{W} \cup \mathcal{B}$; $\quad \mathcal{C} \leftarrow \{1, \dots, n\} \setminus \mathcal{P}$ ▷ separate protected and candidate tokens

% Stage 1: estimate future decoding queries

for $m = 1, \dots, M$ **in parallel do**

  $Y^{(m)} \sim q(\cdot \mid X)$, $|Y^{(m)}| = T$; store $\mathcal{Q}_{\mathrm{look}}^{(m),l,h}\ \forall (l,h)$, $\{c_u^{(m)}, H_u^{(m)}\}_{u=1}^{T}$ ▷ sample a lookahead; store its queries and per-step c_u (Eq. equation 3), H_u

end for

$\mathbf{D} \leftarrow \textsc{Disagree}\big(\{\mathcal{Q}_{\mathrm{look}}^{(m),l,h}\}_{m=1}^{M}\big)$ by Eq. equation 14; $\hat{\mathbf{D}} \leftarrow \mathrm{zscore}(\mathbf{D})$ if $M \geq 3$ else $\mathbf{0}$ ▷ disagreement from the stored queries; no extra decoding

$\bar{c}_m \leftarrow \frac{1}{T}\sum_u c_u^{(m)}$, $\bar{H}_m \leftarrow \frac{1}{T}\sum_u H_u^{(m)}$, $R_m \leftarrow \beta \bar{c}_m - \gamma \bar{H}_m - \lambda_D \hat{D}_m \quad \forall m$ ▷ means, not sequence totals (Eq. equation 4)

$\boldsymbol{\omega} \leftarrow \mathrm{softmax}(\mathbf{R}) \in \Delta^{M-1}$, $\mathbf{R} = (R_m)_{m=1}^{M}$ ▷ convert reliability scores into trajectory weights

$\mathcal{Q}_{\mathrm{score}}^{l,h} \leftarrow \mathcal{Q}_{\mathcal{W}}^{l,h} \cup \bigcup_{m=1}^{M} \mathcal{Q}_{\mathrm{look}}^{(m),l,h}$ ▷ combine prompt and lookahead queries

% Stage 2: score the impact of removing each token

for layer $l = 1, \dots, L$ **and** head $h = 1, \dots, H$ **do**

  for candidate $i \in \mathcal{C}$ **and** query $\mathbf{q} \in \mathcal{Q}_{\mathrm{score}}^{l,h}$ **do**

    $(a_{q,i}^{l,h}, \mathbf{o}_q^{l,h}) \leftarrow \mathrm{Attn}(\mathbf{q}, \mathbf{K}^{l,h}, \mathbf{V}^{l,h})$ ▷ compute attention weight and output

    $\Delta \mathbf{o}_{q,i}^{l,h} \leftarrow \dfrac{a_{q,i}^{l,h}}{1 - a_{q,i}^{l,h} + \epsilon}\big(\mathbf{v}_i^{l,h} - \mathbf{o}_q^{l,h}\big)$

    $e_{q,i}^{l,h} \leftarrow \left\| \mathbf{W}_{l,h}^{O} \Delta \mathbf{o}_{q,i}^{l,h} \right\|_2^2$ ▷ measure the effect of removing token i

  end for

  $S_i^{l,h} \leftarrow \rho(\mathcal{E}_{i,\mathcal{W}}^{l,h}) + \lambda_L \sum_{m=1}^{M} \omega_m \rho(\mathcal{E}_{i,m}^{l,h}) \quad \forall i \in \mathcal{C}$ ▷ combine prompt and weighted lookahead importance

  % Stage 3: allocate cache budget and retain tokens

  $\{B_{l,h}\}_{h=1}^{H} \leftarrow \mathrm{Alloc}\big(B_l, \{E_{l,h}\}_{h=1}^{H}; \tau\big)$ ▷ divide the layer budget across heads

  for head $h = 1, \dots, H$ **do**

    $\mathcal{R}^{l,h} \leftarrow \mathrm{Select}(\mathbf{S}^{l,h}, B_{l,h})$ ▷ keep the highest-scoring tokens

    $\mathcal{R}_{\mathrm{final}}^{l,h} \leftarrow \mathcal{P} \cup \mathcal{R}^{l,h}$

    $(\mathbf{K}_\star^{l,h}, \mathbf{V}_\star^{l,h}) \leftarrow \mathrm{Gather}(\mathbf{K}^{l,h}, \mathbf{V}^{l,h}, \mathcal{R}_{\mathrm{final}}^{l,h})$ ▷ build the compressed cache

  end for

end for

discard lookahead states

return $(\mathbf{K}^\star, \mathbf{V}^\star)$ ▷ use the compressed cache for final decoding

atypicality zero-mean conditional on the visited context so that uncertainty can be parameterized explicitly through γ.

Proposition 1 (nesting). Because $\frac{1}{T}\sum_u \log q(y_u^{(m)} \mid h_u^{(m)}) = \bar{c}_m - \bar{H}_m$, the earlier score $\frac{1}{T}\sum_u \log q - \lambda_H \bar{H}_m - \lambda_D \hat{D}_m$ is exactly $\bar{c}_m - (1 + \lambda_H)\bar{H}_m - \lambda_D \hat{D}_m$. Thus uniform weighting is $(\beta, \gamma, \lambda_D) = (0, 0, 0)$; the earlier "log-probability only" setting is $(1, 1, 0)$, not a no-entropy control; LORE-KV /E is $(1, 1.3, 0)$; and LORE-KV /D is $(1, 1, 0.3)$. This is also why LORE-KV /E and LORE-KV /D are variant settings rather than a single-factor ablation.

Variance and scale. For $u < v$, c_u is h_v-measurable and $\mathbb{E}[c_u c_v] = \mathbb{E}[c_u\, \mathbb{E}[c_v \mid h_v]] = 0$. Hence

$$\mathrm{Var}(\bar{c}_m) = \frac{1}{T^2}\sum_{u=1}^{T} \mathbb{E}[V_u], \qquad V_u = \mathrm{Var}_{y \sim q(\cdot \mid h_u)}[\log q(y \mid h_u)], \tag{13}$$

where V_u is the varentropy. The scale of $\bar{c}_m$ can therefore vary across prompts. The optional normalization $\tilde{c}_m = \bar{c}_m / \sqrt{\bar{V}_m / T}$, with $\bar{V}_m = \frac{1}{T} \sum_u V_u^{(m)}$, gives unit scale by construction; it is evaluated only as an ablation and is not part of the default method.

Query-space disagreement. Predictive entropy is defined in token-probability space, whereas the eviction score consumes query directions. We therefore define the disagreement of particle m as the mean cosine distance to the other particles at matched decoding positions, averaged over lookahead steps, layers, and query heads:

$$D_m = \frac{1}{M-1} \sum_{r \neq m} \frac{1}{TLH} \sum_{u=1}^{T} \sum_{l=1}^{L} \sum_{h=1}^{H} \left[1 - \cos\left(\mathbf{q}_u^{(m),l,h}, \mathbf{q}_u^{(r),l,h} \right) \right]. \tag{14}$$

Queries compared at step u occupy the same absolute position $n + u$ and therefore share the same rotary phase. We standardize disagreement across particles,

$$\hat{D}_m = \frac{D_m - \bar{D}}{\mathrm{sd}(D)}, \qquad \bar{D} = \frac{1}{M} \sum_{r=1}^{M} D_r. \tag{15}$$

For $M = 1$, disagreement is undefined; for $M = 2$, $D_1 = D_2$ by symmetry and the standard deviation is zero. We therefore set $\hat{D}_m \equiv 0$ for $M \leq 2$, so disagreement can affect weights only for $M \geq 3$. For ragged trajectories, $\bar{c}_m$ and $\bar{H}_m$ use each particle's available steps, whereas D_m is computed on the common prefix required for step-wise alignment (Appendix D.1). The disagreement computation reuses the stored lookahead queries and requires no additional decoding (Appendix B.4).

The entropy and disagreement coefficients are not directly comparable numerically because $\bar{H}_m$ is measured in nats while $\hat{D}_m$ is standardized. We therefore calibrate λ_D using the realized across-particle spread of the two penalties rather than equating their raw coefficients.

Softmax parameterization and estimator interpretation. No separate particle-weight temperature is introduced: multiplying Eq. equation 4 by a concentration parameter η is equivalent to rescaling $(\beta, \gamma, \lambda_D)$, so η is not separately identifiable.

With uniform weights, the lookahead term is the ordinary Monte-Carlo estimate of expected deletion effect under continuations sampled from $p_\theta(\cdot \mid X)$. When $\lambda_D = 0$, each weight depends only on its own trajectory and the normalized weighted estimate has target

$$\pi(Y) \propto p_\theta(Y \mid X) \exp\left(\beta \bar{c}(Y) - \gamma \bar{H}(Y) \right).$$

To first order,

$$\mathbb{E}_\pi[f] = \mathbb{E}_{p_\theta}[f] + \beta \, \mathrm{Cov}_{p_\theta}(f, \bar{c}) - \gamma \, \mathrm{Cov}_{p_\theta}(f, \bar{H}) + O(\|(\beta, \gamma)\|^2).$$

Because $\mathbb{E}_{p_\theta}[\bar{c}] = 0$, centering removes a zeroth-order shift from branch atypicality; it does not imply an unbiased finite-β estimator or zero higher-order covariance. When $\lambda_D > 0$, $\hat{D}_m$ depends on the full particle set, so the weights form an interacting-particle estimator rather than finite-M importance weights for a fixed target law. A fixed disagreement functional is recovered only asymptotically as M grows.

Failure modes and safeguards. LORE-KV /E can assign substantial weight to a confidently wrong continuation because low predictive entropy does not guarantee correctness. LORE-KV /D can suppress such a trajectory when its query path is atypical relative to the sampled ensemble, but it does not detect a collectively wrong consensus and is unreliable when too few particles are available. Step-wise cosine disagreement can also overstate differences between semantically similar continuations with different tokenizations or lengths. These limitations motivate soft weighting, the bounded contribution of the lookahead term through λ_L, and the use of LORE-KV /E as the default until the disagreement variant is supported by complete end-to-end evidence. Figures 6 and 7 report the empirical relation between these signals and oracle particle utility.

B.4 Computational Overhead

At a fixed cache budget B, all methods retain the same number of KV pairs and therefore have matched steady-state cache memory and final-decoding cost; LORE-KV trades additional one-time

wall-clock computation for more reliable token retention under the same memory budget. Specifically, the M conditionally independent lookahead particles are generated as a single batched continuation of length T from the shared prompt cache, and scoring is restricted to the $w + MT \ll n$ prompt-window and lookahead queries. The reliability term costs nothing on top of this: the entropy of LORE-KV /E is a by-product of the sampling logits, and the disagreement of LORE-KV /D is one $M \times M$ cosine reduction per layer over query tensors that are already materialized, i.e. $O(M^2LHTd)$ arithmetic with no additional memory and no additional decoding, which is negligible beside the MT decode steps that produced those tensors. The two variants therefore have the same wall-clock profile, and Table 4 applies to both. Table 4 quantifies this accuracy-latency trade-off: LORE-KV /E requires approximately $1.46\times$ to $2.77\times$ the per-sample wall-clock time of AnDPro across the reported settings, while preserving the same B-token cache for final decoding; the same numbers apply to LORE-KV /D by the cost argument above. This additional pre-generation computation provides a more representative future-query reference and improves retention quality, particularly under aggressive compression where preserving task-relevant evidence is most difficult. Importantly, the overhead does not increase monotonically with B, consistent with its primary source being the fixed (M, T) lookahead and scoring procedure rather than increased cache memory. Several controls further separate the improved retention criterion from additional computation alone: **(i)** all methods use the same B, matching steady-state memory and per-token final-decoding cost; **(ii)** despite fixed lookahead compute, the RULER gain over AnDPro decreases from 5.05 to 0.42 points on Mistral and from 3.52 to 0.43 on LLaMA as B increases from 128 to 1024 (Table 3), query coverage saturates at small M (Figure 4), and per-example gains increase with prompt-to-future query mismatch (Figure 2); **(iii)** $\lambda_L = 0$ (equivalently $M = 0$) removes lookahead and recovers the prompt-window criterion (Table 10); and **(iv)** a compute-matched control supplying the same lookahead continuations to Lookahead Q-Cache-style scoring directly equalizes test-time compute: the end-to-end result is **LAQ++(8)** (Wang et al., 2025b), reported in Tables 1–7 and Section 3.3, where LORE-KV /E remains ahead on average despite the matched compute; the corresponding score-level analysis is reported in Appendix E.3.

B.5 Hyperparameters and Switches

The core method exposes seven quantities: the observation window w, the lookahead count M and length T, the three reliability coefficients β (typicality), γ (entropy) and λ_D (trajectory disagreement), and the mixing coefficient λ_L; the constant ϵ only guards the renormalization in Eq. equation 6. The plug-ins add α, κ (robust aggregation), τ (adaptive head budgets) and λ_k, λ_v (diversity selection); we write the robust-aggregation top-fraction as κ because β now names the typicality coefficient of Eq. equation 4. Each plug-in recovers its default exactly at zero: $\lambda_L{=}0 \Leftrightarrow M{=}0$ (no lookahead), $\alpha{=}0$ (mean aggregation), $\tau{=}0$ (uniform budgets), $\lambda_k{=}\lambda_v{=}0$ (top-score selection), which is what makes the block-level ablation protocol well-posed as detailed in Table 5. The reliability block is different in kind: $(\beta, \gamma, \lambda_D)$ is a *plane* that all three regimes—LORE-KV /E at $(1, 1.3, 0)$, LORE-KV /D at $(1, 1, 0.3)$, and log-probability only at $(1, 1, 0)$—occupy as distinct points (Proposition 1), rather than a set of switches, so a run of any of them reproduces the others by changing coefficients, with no re-decoding when the particles are already captured. Note that the no-weighting control is $(0, 0, 0)$, and the no-*entropy* control is $(1, 0, 0)$; neither coincides with the earlier "log-probability only" setting, which is $(1, 1, 0)$.

There is no particle-weight temperature η: it is not identifiable alongside $(\beta, \gamma, \lambda_D)$ (Section 2.3) and has been absorbed into them.

C Plug-In Components and the Ablation Protocol

The core method deliberately keeps all downstream operations simple, using mean aggregation, uniform head-wise budgets, and top-score selection. Building on these defaults, we consider three optional plug-ins motivated by established designs in prior work. Each is controlled by a separate switch, allowing its incremental effect to be isolated relative to the fixed core configuration. The plug-ins act on ρ, Alloc and Select and are therefore orthogonal to the reliability coefficients: the executed rows sit on the LORE-KV /E core, and their LORE-KV /D counterparts—the same switch settings at $(\beta, \gamma, \lambda_D) = (1, 1, 0.3)$—are the – block of Table 10. Results are reported in Table 10 for LongBench, Table 11 for RULER, and Table 10 for the complete configuration grid.

Table 5: LORE-KV hyperparameters and default settings.

Symbol	Meaning	Default
w	prompt-end observation-window size	32
M	number of lookahead particles	4
T	lookahead particle length	8
β	typicality coefficient in Eq. equation 4	1.0
γ	entropy coefficient in Eq. equation 4	1.3 (LORE-KV /E) / 1 (LORE-KV /D)
λ_D	disagreement coefficient in Eq. equation 4	0 (LORE-KV /E) / 0.3 (LORE-KV /D)
λ_L	lookahead mixing coefficient in Eq. equation 8	1.0
ϵ	renormalization guard in Eq. equation 6	10^{-6}
κ, α	robust aggregation (Appendix C)	1, 0
τ	adaptive head budgets (Appendix C)	0
λ_k, λ_v	diversity selection (Appendix C)	0

A. Robust aggregation (α, κ). A plain mean can suppress a token that is important for only a small subset of future queries, motivating a robust aggregation strategy in the spirit of DefensiveKV (Feng et al., 2025). We therefore replace the mean with a top-κ average, $\mathrm{TopAvg}_\kappa(\mathcal{E}) = \frac{1}{K_\kappa}\sum_{e\in\mathrm{TopK}(\mathcal{E},K_\kappa)} e$, where $K_\kappa = \lceil \kappa|\mathcal{E}|\rceil$, and interpolate it with the standard mean as $\rho(\mathcal{E}) = \alpha\,\mathrm{TopAvg}_\kappa(\mathcal{E}) + (1-\alpha)\,\mathrm{Mean}(\mathcal{E})$, with $\alpha \in [0,1]$. (This top-fraction was written β in an earlier draft; it is renamed κ here because β now denotes the typicality coefficient of Eq. equation 4.) Substituting $\rho(\mathcal{E})$ for the mean in Eq. equation 8 yields the robust variant; $\alpha = 0$ exactly recovers the core configuration, while larger α increases sensitivity to high-importance query subsets and provides an additional safeguard against a misleading particle.

B. Adaptive head-wise budgets (τ). The core method splits the layer budget evenly across heads. Adaptive allocation is inherited from Ada-KV (Feng et al., 2026) and the layer-wise scheme of PyramidKV (Cai et al., 2024); our one variation is that the allocation mass is each head's total future-query perturbation,

$$E_{l,h} = \sum_{i\in\mathcal{C}} S_i^{l,h}, \tag{16}$$

which is what Alloc of Eq. equation 9 consumes. Its $\tau > 0$ branch is

$$B_{l,h} = \left\lfloor B_l \frac{(E_{l,h}+\epsilon)^\tau}{\sum_{h'=1}^{H}(E_{l,h'}+\epsilon)^\tau} \right\rfloor, \tag{17}$$

with $\tau \geq 0$ setting how aggressively capacity shifts toward high-impact heads; $\tau = 0$ returns $\lfloor B_l/H \rfloor$. This separates "better scores" from "better placement of a fixed budget." Figure 9 supplies the premise: both $E_{l,h}$ and the benefit of the future-query term are distributed very unevenly across depth and heads.

C. Diversity-aware selection (λ_k, λ_v). Top-$B_{l,h}$ selection can in principle retain many near-duplicate keys or values. As an ablation we make Select a greedy rule inspired by redundancy-aware selection such as GraphKV (Li et al., 2025b): starting from $\mathcal{R}^{l,h} = \emptyset$, each newly selected token is

$$i^\star = \arg\max_{i\in\mathcal{C}\setminus\mathcal{R}^{l,h}} \left[S_i^{l,h} - \lambda_k \max_{j\in\mathcal{R}^{l,h}} \cos(\mathbf{k}_i^{l,h}, \mathbf{k}_j^{l,h}) - \lambda_v \max_{j\in\mathcal{R}^{l,h}} \cos(\mathbf{v}_i^{l,h}, \mathbf{v}_j^{l,h}) \right], \tag{18}$$

repeated until $|\mathcal{R}^{l,h}| = B_{l,h}$; $\lambda_k = \lambda_v = 0$ recovers plain top-$B_{l,h}$. The known failure mode is that a cosine penalty can discard evidence that is repeated but genuinely important, as when several supporting passages mention the same entity. The penalty acts on the perturbation score rather than on raw key/value features, so a duplicate that still drives a large future-query perturbation survives; and λ_k, λ_v are small, so redundancy only breaks ties between tokens of comparable importance. The output-perturbation score may already suppress redundancy on its own, since a duplicated token whose deletion is compensated by its near-copy has a small $\Delta\mathbf{o}_{q,i}^{l,h}$ by construction.

Ablations of the core itself. Three axes stress the claimed contribution directly. **(i) Lookahead budget:** $M \in \{0,1,2,4,8\}$ at fixed T=8, where M=0 (equivalently λ_L=0) collapses to a prompt-window output-perturbation method; saturation at small M is prediction P2. Varying T is a planned

axis we have not yet run. **(ii) Reliability weighting:** the $(\beta, \gamma, \lambda_D)$ cells of Table 10, which price each channel of Eq. equation 4 separately. At $\lambda_D{=}0$ these are the missing uniform control $(0, 0, 0)$, the no-entropy control $(1, 0, 0)$, the stronger-entropy probe $(1, 2, 0)$ and the two published cells $(1, 1, 0)$ and $(1, 1.3, 0)$; at $\lambda_D{>}0$ they add $(1, 1, 0.3)$ and the single-factor cell $(1, 0, 0.3)$ that puts disagreement in place of entropy rather than on top of it. The two cells the method itself ships as variants—LORE-KV /E and LORE-KV /D—are Appendix F; the rest are controls. All cells run at uniform head budgets and top-K selection, everything else held fixed, and re-weight nested prefixes of one 8-particle draw per prompt so no cell differs by test-time compute. **(iii) Perturbation objective:** we replace Eq. equation 7 with accumulated attention mass, the unprojected perturbation norm $\|\Delta \mathbf{o}_{q,i}^{l,h}\|_2^2$, under the identical lookahead queries, providing the compute-matched control described in Appendix B.4.

D Experimental Setup and Additional Results

D.1 Protocol, Datasets and Baselines

We evaluate LORE-KV on LongBench, RULER, and Needle-in-a-Haystack while testing three mechanism-level predictions: gains should be larger under tighter budgets and longer contexts **(P1)**, saturate after a few lookahead trajectories **(P2)**, and increase with prompt window future-query mismatch **(P3)**. LongBench covers 16 English tasks across QA, summarization, few-shot learning, synthetic retrieval, and code, with input lengths from $1{,}235$ to $18{,}409$ tokens; Table 6 reports task metrics, average lengths, and sample counts. RULER provides a controlled complement by evaluating 13 fixed-length tasks, including eight needle variants, variable tracking, common- and frequent-word extraction, and two QA tasks; because its question appears at the prompt end, the observation window is a comparatively strong proxy for future queries. Needle-in-a-Haystack varies target depth and context length to test whether answer-bearing evidence survives eviction.

Baselines and sourcing. Table 1 compares StreamingLLM (Xiao et al., 2024), H2O (Zhang et al., 2023), SnapKV (Li et al., 2024), PyramidKV (Cai et al., 2024), Ada-KV (Feng et al., 2026), CriticalKV (Guo et al., 2024), and AnDPro (Geng et al., 2026), with Full Cache as the uncompressed reference. StreamingLLM through Ada-KV use prompt-side attention statistics, whereas CriticalKV and AnDPro incorporate value- or output-aware information while remaining prompt-side. LongBench results with Mistral follow the published protocol of Geng et al. (2026); CriticalKV, AnDPro, and LORE-KV /E are re-run under one implementation for RULER, Needle-in-a-Haystack, and LongBench with Llama; LORE-KV /D runs in that same implementation and is pending at submission time. Lookahead Q-Cache (Wang et al., 2025b) is additionally included as the compute-matched end-to-end baseline **LAQ++(8)** in Tables 1 and 7 (Section 3.1) and, where available, in Section 3.3; the corresponding score-level compute-matched control remains a separate diagnostic, analyzed in Appendix E.3.

Implementation. Our implementation builds on the Ada-KV framework and uses a single NVIDIA H100 80 GB GPU. All methods use the same retained cache size at a fixed budget, and adjacent-token grouping follows SnapKV and Ada-KV. Unless stated otherwise, LORE-KV uses $M{=}4$, $T{=}8$, $\lambda_L{=}1.0$, and disables all plugins ($\alpha{=}\tau{=}\lambda_k{=}\lambda_v{=}0$), thereby isolating the score in Eq. equation 8. The reliability coefficients are the only difference between the two variants: $(\beta, \gamma, \lambda_D){=}(1, 1.3, 0)$ for LORE-KV /E and $(1, 1, 0.3)$ for LORE-KV /D.

Lookahead particles are sampled at temperature 1 with no top-k or top-p truncation, so the sampling law is p_θ itself and the centering of Eq. equation 3 is exact. The implementation computes c_u, H_u and the varentropy V_u from the *same* shaped logits the sampler consumed, so a future change of sampling law cannot silently break $\mathbb{E}[c_u \mid h_u] = 0$; entropy and varentropy use masked reductions, since outside a truncated support $q \log q$ would otherwise be $0 \cdot (-\infty)$. Per-step statistics are stored, not only trajectory means, at a cost of MT floats per prompt. Disagreement is computed over every decoder layer (a layer stride is available but unused), from the lookahead query buffers the scoring stage already holds, and must be evaluated before the cache is compressed because compression releases those buffers. If a particle terminates early, the D_m comparison is truncated to the shortest trajectory in the ensemble so that the step-wise pairing in Eq. equation 14 stays valid; $\bar{c}_m$ and $\bar{H}_m$ need no such rule, since each averages over its own particle's steps.

Table 6: LongBench English datasets, metrics, average input lengths, and sample counts (Bai et al., 2024).

Task	Task type	Metric	Avg. len	Language	#Samples
NarrativeQA	Single-Doc. QA	F1	18,409	EN	200
Qasper	Single-Doc. QA	F1	3,619	EN	200
MultiFieldQA-en	Single-Doc. QA	F1	4,559	EN	150
HotpotQA	Multi-Doc. QA	F1	9,151	EN	200
2WikiMultihopQA	Multi-Doc. QA	F1	4,887	EN	200
MuSiQue	Multi-Doc. QA	F1	11,214	EN	200
GovReport	Summarization	Rouge-L	8,734	EN	200
QMSum	Summarization	Rouge-L	10,614	EN	200
MultiNews	Summarization	Rouge-L	2,113	EN	200
TREC	Few-shot Learning	Accuracy	5,177	EN	200
TriviaQA	Few-shot Learning	F1	8,209	EN	200
SAMSum	Few-shot Learning	Rouge-L	6,258	EN	200
PassageCount	Synthetic	Accuracy	11,141	EN	200
PassageRetrieval-en	Synthetic	Accuracy	9,289	EN	200
LCC	Code	Edit Sim	1,235	Python/C#/Java	500
RepoBench-P	Code	Edit Sim	4,206	Python/Java	500

Table 7: LongBench results with Llama-3.1-8B-Instruct across cache budgets. CriticalKV, AnDPro, and LORE-KV are matched runs; other baseline values are from Geng et al. (2026). – denotes runs in progress. LAQ++(8) is a compute-matched extension of Lookahead Q-Cache (Wang et al., 2025b) run under our implementation at $B \in \{128, 256, 512\}$ (not evaluated at $B = 1024$); it is reported for reference and excluded from the / highlighting.

Method	NrtvQA	Qasper	MF-en	HotpotQA	2WikiMQA	Musique	GovReport	QMSum	MultiNews	TREC	TriviaQA	SAMSum	PCount	PRe	Lcc	RB-P	Avg.
Full Cache	30.12	46.60	56.41	58.10	49.01	32.52	34.04	25.37	27.06	73.00	91.90	43.46	7.06	100.00	62.13	51.80	49.29
Budget = 128																	
Budget = 128																	
SnapKV	25.54	31.40	50.99	55.17	45.50	27.88	20.77	23.41	21.05	47.50	90.46	40.76	8.00	99.50	56.88	46.50	43.21
PyramidKV	27.83	32.98	51.45	56.72	44.67	30.54	21.89	22.90	21.67	63.00	90.45	40.06	8.00	**100.00**	55.37	43.79	44.46
Ada-KV	25.06	34.50	52.27	56.29	47.57	29.06	21.17	23.36	21.51	62.00	**92.17**	40.84	8.00	99.50	58.45	48.72	45.03
CriticalKV	26.00	34.96	51.79	56.01	46.53	28.95	21.31	23.14	21.33	62.16	89.51	40.74	7.59	97.89	57.85	48.28	44.63
AnDPro	25.25	35.85	**52.85**	54.35	44.41	27.51	22.82	**23.60**	22.20	69.00	92.00	41.50	7.02	**100.00**	**62.26**	**56.34**	46.06
LAQ++(8) (Wang et al., 2025b)	28.65	10.65	26.04	24.23	21.56	15.67	23.50	23.74	22.75	72.00	91.95	42.19	8.37	94.81	62.18	50.95	38.70
LORE-KV /E (ours)	27.38	37.66	51.14	**57.37**	**48.47**	**30.55**	23.77	**23.60**	**23.14**	68.04	90.46	41.77	7.34	97.90	58.99	50.15	46.11
LORE-KV /D (ours)	**29.00**	**39.20**	52.83	54.76	45.31	30.20	**23.86**	23.48	22.26	**71.00**	92.09	**42.75**	**8.01**	**100.00**	61.63	55.76	**47.01**
Budget = 256																	
SnapKV	26.92	38.19	52.62	57.03	46.75	29.63	23.07	23.92	22.82	58.50	91.83	40.81	7.75	99.50	59.95	48.63	45.50
PyramidKV	28.46	39.36	54.30	56.77	45.50	30.74	23.84	23.88	22.91	69.00	91.09	40.74	**7.88**	99.50	56.94	45.18	46.01
Ada-KV	27.04	39.24	54.41	57.36	47.53	31.35	23.06	23.81	23.43	69.00	**92.50**	41.23	7.67	**100.00**	61.45	49.52	46.79
CriticalKV	25.45	39.60	52.90	55.72	46.05	29.97	23.10	23.77	22.77	68.28	90.48	41.08	7.48	97.55	60.19	48.25	45.79
AnDPro	27.23	42.01	**54.62**	**57.87**	47.43	31.92	23.53	24.22	23.54	71.87	92.37	**41.47**	7.66	99.81	62.45	51.67	47.48
LAQ++(8) (Wang et al., 2025b)	30.25	12.43	26.63	25.77	22.83	18.45	25.07	23.67	23.75	72.00	91.97	43.17	6.93	94.59	64.07	53.38	39.69
LORE-KV /E (ours)	28.26	44.01	53.43	57.73	**48.78**	**32.30**	25.63	**24.93**	**24.10**	71.43	91.03	39.98	7.39	98.52	61.70	51.28	47.53
LORE-KV /D (ours)	**29.16**	**44.86**	54.37	56.29	47.03	32.11	**25.68**	24.86	23.61	**73.07**	91.93	40.52	7.76	99.68	**63.16**	**54.38**	**48.03**
Budget = 512																	
SnapKV	28.40	41.52	55.40	57.66	47.48	30.95	24.92	24.46	24.54	68.50	92.33	41.98	7.33	99.50	61.25	50.39	47.29
PyramidKV	29.15	43.04	55.55	57.71	47.67	31.27	26.05	24.08	24.54	70.50	**92.67**	41.52	7.33	**100.00**	59.96	47.51	47.41
Ada-KV	26.92	42.68	55.20	57.47	48.06	31.27	24.95	24.17	24.40	71.00	92.14	**42.35**	7.33	**100.00**	62.26	50.62	47.55
CriticalKV	26.00	42.53	53.90	56.62	46.76	31.13	24.26	23.76	23.81	70.47	89.56	40.75	6.92	97.20	60.51	49.74	46.50
AnDPro	27.97	44.15	55.49	58.16	48.19	31.92	24.80	24.50	24.77	72.84	91.76	41.99	7.31	99.78	**62.54**	52.00	48.01
LAQ++(8) (Wang et al., 2025b)	29.64	13.22	26.79	27.58	23.49	18.63	26.94	24.04	25.21	72.50	92.25	42.83	8.43	96.25	65.00	53.37	40.39
LORE-KV /E (ours)	29.77	43.61	55.22	**58.32**	**48.68**	**32.73**	27.40	**24.96**	**25.02**	72.65	91.50	40.50	7.46	99.52	60.38	51.25	48.06
LORE-KV /D (ours)	**30.40**	**44.21**	**55.87**	57.31	47.46	32.59	**27.43**	24.91	24.68	**73.79**	92.13	40.88	**7.72**	**100.00**	61.40	**53.42**	**48.39**
Budget = 1024																	
SnapKV	28.32	45.37	**56.81**	58.16	48.28	31.89	26.91	24.58	25.82	70.50	92.22	43.08	6.96	**100.00**	62.46	51.50	48.30
PyramidKV	29.32	44.57	56.14	58.39	48.36	31.93	28.20	23.78	26.00	70.50	92.41	42.46	6.88	**100.00**	61.33	49.12	48.09
Ada-KV	28.52	**46.01**	55.72	57.85	48.10	32.02	26.98	24.72	25.96	72.00	91.97	42.55	6.96	**100.00**	61.98	51.52	48.30
CriticalKV	27.27	44.74	54.24	56.17	46.95	31.65	26.35	23.91	24.93	70.22	89.08	41.00	6.70	96.86	59.85	49.75	46.85
AnDPro	**31.36**	43.42	55.10	55.05	45.93	29.79	28.46	24.70	**26.56**	72.50	91.42	**43.57**	7.39	99.50	**64.86**	**59.01**	48.66
LORE-KV /E (ours)	29.99	44.83	56.19	**58.60**	**49.01**	**33.02**	28.90	**25.49**	26.38	73.00	92.35	42.24	7.50	99.50	61.31	51.60	48.74
LORE-KV /D (ours)	30.27	45.10	56.48	58.15	48.46	32.96	**28.92**	25.47	26.23	**73.51**	**92.63**	42.41	**7.62**	99.86	61.77	52.57	**48.90**

D.2 LongBench with Llama-3.1-8B-Instruct

Table 7 provides the per-dataset Llama-3.1-8B-Instruct results. CriticalKV, AnDPro, and LORE-KV /E are directly comparable because they use the same implementation, whereas SnapKV, PyramidKV, and Ada-KV are published results from Geng et al. (2026). LORE-KV /E has the highest 16-task average at all four budgets, although its margin over AnDPro is small; the LORE-KV /D rows are – at every budget and use the same implementation and particles, differing only in (γ, λ_D).

At B=128, LORE-KV /E shows the largest gains over AnDPro on multi-hop and long-document QA, with smaller improvements on summarization, while performance drops mainly on code completion and a few retrieval/few-shot tasks. This trend is consistent with the per-dataset results in

Table 8: RULER per-task results with Mistral-7B-Instruct-v0.2. CWE/FWE denote common-/frequent-word extraction; MK/MV/MQ/S denote multi-key/multi-value/multi-query/single-needle variants; VT denotes variable tracking.

B	Method	CWE	FWE	MK1	MK2	MK3	MQ	MV	S1	S2	S3	QA-1	QA-2	VT	Avg.
						Context length = 4K									
128	CriticalKV	9.40	40.20	89.20	70.40	0.00	0.80	17.40	89.20	96.00	0.00	82.20	59.40	15.60	43.83
128	AnDPro	17.20	44.20	98.60	87.60	**0.40**	**37.40**	**46.40**	99.60	**99.80**	0.00	82.60	61.20	38.00	54.85
128	**LORE-KV /E**	**29.62**	**61.27**	**99.80**	99.60	0.00	19.40	36.50	**100.00**	**99.80**	**4.00**	84.00	61.60	**63.56**	**58.40**
128	**LORE-KV /D**	29.18	60.80	**99.80**	**99.80**	0.00	19.00	36.30	**100.00**	**99.80**	3.80	**84.20**	**62.00**	63.40	58.31
256	CriticalKV	24.20	70.80	93.20	85.00	17.20	70.00	60.20	93.20	97.40	9.00	83.20	60.80	66.40	63.89
256	AnDPro	29.20	73.40	99.40	96.20	**17.40**	**93.80**	**79.00**	**100.00**	**99.80**	9.00	83.40	62.00	81.00	71.05
256	**LORE-KV /E**	35.20	82.40	98.60	98.60	15.80	81.60	71.80	98.60	98.40	10.00	82.80	60.80	**95.00**	**71.51**
256	**LORE-KV /D**	**41.52**	**84.07**	**99.60**	**100.00**	4.80	49.15	59.25	**100.00**	**99.80**	**32.80**	**84.20**	**62.40**	89.44	69.77
512	CriticalKV	43.60	84.00	95.80	92.80	69.20	83.60	78.60	96.00	98.40	54.80	84.20	61.60	87.80	79.26
512	AnDPro	46.40	85.40	**99.40**	99.40	**69.40**	**97.40**	**89.60**	**100.00**	**99.80**	54.80	84.40	**62.20**	96.20	83.42
512	**LORE-KV /E**	50.80	**91.40**	**99.40**	99.60	68.80	91.00	86.00	99.60	99.40	55.80	84.40	62.00	**99.80**	**83.69**
512	**LORE-KV /D**	**60.22**	89.53	**99.40**	**100.00**	35.40	81.45	78.65	**100.00**	**99.80**	**72.40**	**85.40**	61.80	97.72	81.67
1024	CriticalKV	76.40	89.00	97.60	96.60	**89.20**	90.20	84.60	98.00	99.00	93.80	**85.00**	61.20	93.60	88.78
1024	AnDPro	78.00	89.80	**99.40**	**100.00**	**89.20**	**97.40**	**90.40**	**100.00**	**99.80**	93.80	**85.00**	**61.60**	98.20	90.97
1024	**LORE-KV /E**	80.60	**93.20**	**99.40**	99.80	89.00	93.80	88.40	99.80	99.60	**94.40**	**85.00**	61.40	**100.00**	**91.11**
1024	**LORE-KV /D**	**87.72**	91.53	99.20	**100.00**	73.20	94.00	86.70	**100.00**	**99.80**	91.60	84.80	61.00	98.88	89.88
						Context length = 8K									
128	CriticalKV	3.20	26.40	75.20	59.60	0.00	0.00	6.00	92.60	88.40	0.00	75.40	58.80	18.20	38.75
128	AnDPro	7.40	29.60	94.60	82.80	**0.40**	**19.60**	35.40	**100.00**	97.20	0.00	74.40	59.40	50.00	50.06
128	**LORE-KV /E**	**19.62**	**57.73**	96.40	98.40	0.00	12.90	35.20	**100.00**	**100.00**	**1.20**	**78.20**	61.40	**63.92**	**55.77**
128	**LORE-KV /D**	19.16	56.47	**96.80**	**98.60**	0.00	13.10	**35.95**	**100.00**	**100.00**	0.80	77.80	**62.00**	**63.92**	55.74
256	CriticalKV	11.40	46.40	83.80	77.20	4.00	68.80	52.20	95.20	94.00	2.60	**77.20**	59.80	65.00	56.74
256	AnDPro	14.20	48.40	96.40	92.20	**4.20**	**81.60**	**71.20**	**100.00**	99.80	2.60	76.60	60.20	85.60	64.08
256	**LORE-KV /E**	19.80	63.80	95.60	**98.20**	2.20	75.80	69.40	98.20	98.20	1.60	77.00	59.60	**92.40**	**65.52**
256	**LORE-KV /D**	**29.12**	**69.60**	**96.80**	**98.20**	1.40	34.15	55.60	**100.00**	**100.00**	**25.00**	77.00	**61.40**	91.00	64.56
512	CriticalKV	19.80	67.00	88.80	88.60	26.60	86.60	72.40	97.20	96.60	19.60	**78.20**	57.80	81.60	67.75
512	AnDPro	21.40	68.20	96.20	97.40	**26.80**	**94.00**	**83.60**	**100.00**	**100.00**	19.60	77.80	58.00	93.80	72.06
512	**LORE-KV /E**	24.80	**77.20**	95.80	**99.00**	25.60	90.80	82.60	99.00	99.00	19.00	**78.20**	57.80	**97.80**	**72.82**
512	**LORE-KV /D**	**37.22**	73.07	**96.40**	98.60	9.40	65.15	72.25	**100.00**	**100.00**	**54.80**	76.60	**60.40**	94.48	72.18
1024	CriticalKV	64.60	80.40	94.00	94.40	**68.00**	91.80	81.60	98.60	98.20	**76.20**	**81.80**	59.40	90.60	83.05
1024	AnDPro	65.40	81.20	**97.80**	99.00	**68.00**	**95.80**	**87.40**	**100.00**	**100.00**	**76.20**	81.60	**59.60**	97.00	85.31
1024	**LORE-KV /E**	67.60	**86.40**	97.60	99.40	67.40	94.00	87.00	99.40	99.40	75.80	**81.80**	59.40	**99.40**	**85.74**
1024	**LORE-KV /D**	**74.72**	84.73	97.40	**99.60**	51.60	94.20	85.30	99.60	99.60	73.00	81.60	59.00	98.28	84.51
						Context length = 16K									
128	CriticalKV	0.00	44.20	62.00	50.20	0.00	0.00	0.00	92.60	83.80	0.00	73.00	51.40	11.80	36.08
128	AnDPro	0.00	46.40	82.00	73.80	0.00	0.00	**24.00**	**100.00**	92.80	0.00	72.20	52.40	44.00	45.20
128	**LORE-KV /E**	**12.00**	**71.60**	83.60	88.40	0.00	0.00	21.20	**100.00**	**94.80**	**1.80**	**75.40**	54.00	**60.80**	**51.05**
128	**LORE-KV /D**	11.55	70.73	**83.80**	**88.60**	0.00	0.00	21.48	**100.00**	**94.80**	1.50	75.30	**54.50**	60.72	51.00
256	CriticalKV	5.60	63.80	70.80	67.80	**1.40**	**49.60**	44.20	95.20	89.60	1.00	75.00	52.60	58.80	51.95
256	AnDPro	5.60	65.40	83.80	83.20	**1.40**	**49.60**	**60.00**	**100.00**	95.40	1.00	74.40	53.20	79.80	57.91
256	**LORE-KV /E**	11.80	79.40	83.80	91.00	0.20	42.60	57.20	99.00	95.60	1.20	75.40	53.20	**88.80**	**59.94**
256	**LORE-KV /D**	**19.62**	**83.14**	**84.90**	**91.70**	0.00	5.55	44.03	**100.00**	**97.20**	**24.30**	**76.10**	**54.90**	85.32	58.98
512	CriticalKV	14.60	80.80	78.60	81.20	**24.60**	**68.40**	65.40	97.20	93.00	18.40	**76.40**	52.00	76.80	63.65
512	AnDPro	14.60	81.80	86.20	90.20	**24.60**	**68.40**	**74.60**	**100.00**	96.40	18.40	76.00	52.40	89.00	67.12
512	**LORE-KV /E**	18.40	**90.20**	86.00	**94.60**	23.60	64.60	73.20	99.20	96.40	18.20	**76.40**	52.20	**94.60**	**68.28**
512	**LORE-KV /D**	**22.04**	83.93	**89.00**	88.00	2.80	46.60	60.70	**100.00**	**99.40**	**34.20**	76.00	**53.00**	89.04	64.98
1024	CriticalKV	61.00	89.40	87.20	89.60	**66.60**	**79.00**	76.80	98.60	95.80	**75.40**	**80.60**	55.80	87.40	80.25
1024	AnDPro	61.00	89.80	**91.20**	94.40	**66.60**	**79.00**	**81.60**	**100.00**	97.60	**75.40**	80.40	**56.00**	93.80	82.06
1024	**LORE-KV /E**	63.20	**94.60**	**91.20**	97.00	66.20	76.80	80.60	99.60	97.60	**75.40**	**80.60**	**56.00**	**97.00**	**82.75**
1024	**LORE-KV /D**	**70.32**	92.93	91.00	**97.20**	50.40	77.00	78.90	99.80	**97.80**	72.60	80.40	55.60	95.88	81.53

Table 9: RULER per-task results with Llama-3.1-8B-Instruct. Abbreviations follow Table 8.

B	Method	CWE	FWE	MK1	MK2	MK3	MQ	MV	S1	S2	S3	QA-1	QA-2	VT	Avg.
						Context length = 4K									
128	CriticalKV	9.40	22.00	91.20	78.60	0.00	55.80	59.40	91.20	96.40	0.00	86.20	58.00	11.20	50.72
128	AnDPro	16.00	25.40	99.20	93.20	**0.20**	**86.80**	**84.00**	**100.00**	99.60	0.00	86.60	59.60	30.20	60.06
128	**LORE-KV /E**	**29.82**	**46.67**	**100.00**	95.80	0.00	51.55	66.95	**100.00**	**99.80**	25.40	**88.20**	62.00	69.36	**64.27**
128	**LORE-KV /D**	28.88	46.07	99.80	**96.40**	0.00	50.55	67.20	**100.00**	**99.80**	**25.80**	88.00	**62.60**	**69.56**	64.20
256	CriticalKV	23.80	56.80	94.20	90.00	16.80	79.40	83.60	93.80	97.20	8.80	85.80	59.40	62.60	65.55
256	AnDPro	28.00	59.00	**99.40**	99.60	**17.00**	**99.60**	**99.60**	**99.60**	**99.20**	8.80	**86.00**	**60.40**	74.80	71.62
256	**LORE-KV /E**	**35.00**	**69.00**	98.60	**100.00**	15.60	86.80	92.00	98.20	97.80	10.20	85.60	59.40	90.60	**72.22**
256	**LORE-KV /D**	34.06	68.40	98.40	**100.00**	15.60	85.80	92.25	98.20	97.80	**10.60**	85.40	60.00	**90.80**	72.10
512	CriticalKV	43.00	74.20	96.00	93.60	68.60	87.40	89.80	95.80	97.60	54.20	85.40	60.20	84.40	79.25
512	AnDPro	45.40	75.40	98.80	99.00	**68.80**	**99.00**	**99.00**	**99.00**	**98.60**	54.20	85.60	60.80	91.60	82.71
512	**LORE-KV /E**	**49.80**	**81.40**	**99.00**	99.40	68.40	92.60	95.40	98.80	98.40	55.40	**85.80**	60.80	95.40	**83.12**
512	**LORE-KV /D**	48.86	80.80	98.80	**100.00**	68.40	91.60	95.65	98.80	98.40	**55.80**	85.60	**61.40**	**95.60**	83.05
1024	CriticalKV	75.40	82.40	96.80	96.00	88.00	92.60	93.80	97.00	98.00	92.60	85.00	60.00	91.00	88.35
1024	AnDPro	86.00	89.60	**99.80**	**100.00**	**98.40**	**99.80**	**99.80**	**100.00**	**100.00**	95.60	**87.80**	62.60	94.60	93.38
1024	**LORE-KV /E**	**88.80**	**93.40**	**99.80**	99.80	98.20	96.20	97.80	99.80	99.80	96.40	**87.80**	62.40	96.60	**93.60**
1024	**LORE-KV /D**	87.86	92.80	99.60	**100.00**	98.20	95.20	98.05	99.80	99.80	**96.80**	87.60	**63.00**	**96.80**	93.50
						Context length = 8K									
128	CriticalKV	8.20	13.80	82.80	73.80	0.00	68.20	58.60	93.60	92.00	0.00	81.80	59.40	2.80	48.85
128	AnDPro	11.80	16.60	99.20	93.60	0.00	**84.80**	83.40	99.80	99.40	0.00	81.00	60.00	29.80	58.42
128	**LORE-KV /E**	**33.20**	**39.00**	**100.00**	**100.00**	**8.20**	36.60	87.20	**100.00**	**100.00**	9.40	**94.40**	70.80	45.80	**63.43**
128	**LORE-KV /D**	32.26	38.40	99.80	**100.00**	**8.20**	35.60	**87.45**	**100.00**	**100.00**	**9.80**	94.20	**71.40**	**46.00**	63.32
256	CriticalKV	15.00	36.80	88.60	86.80	3.60	88.60	83.40	95.40	94.60	**2.40**	**81.80**	59.80	52.80	60.74
256	AnDPro	17.20	38.60	**99.40**	99.60	**3.80**	**99.60**	**99.60**	**99.40**	**99.60**	**2.40**	81.20	60.20	70.20	66.98
256	**LORE-KV /E**	**23.80**	**55.80**	98.60	**100.00**	2.00	93.60	97.80	97.60	98.00	1.60	**81.80**	59.80	78.00	**68.34**
256	**LORE-KV /D**	22.86	55.20	98.40	**100.00**	2.00	92.60	98.05	97.60	98.00	2.00	81.60	**60.40**	**78.20**	68.22
512	CriticalKV	22.00	60.20	91.40	91.80	26.00	92.80	89.60	96.80	96.40	**19.40**	80.60	57.40	72.80	69.02
512	AnDPro	23.40	61.20	**97.60**	99.00	**26.20**	**99.00**	**99.00**	**99.00**	**99.00**	**19.40**	80.20	57.60	83.00	72.58
512	**LORE-KV /E**	**27.00**	**70.60**	97.20	**100.00**	25.20	95.80	98.20	98.20	98.20	19.00	**80.80**	57.60	87.20	**73.46**
512	**LORE-KV /D**	26.06	70.00	97.00	**100.00**	25.20	94.80	98.45	98.20	98.20	**19.40**	80.60	**58.20**	**87.40**	73.35
1024	CriticalKV	**65.20**	75.80	94.80	94.80	66.80	95.40	93.80	97.60	97.20	75.20	82.60	58.80	84.40	83.26
1024	AnDPro	45.00	81.60	**100.00**	99.80	**76.40**	**99.60**	**99.80**	**100.00**	**100.00**	**77.20**	83.40	61.80	92.60	85.94
1024	**LORE-KV /E**	47.20	**87.20**	99.80	**100.00**	75.80	97.80	99.20	99.40	99.40	76.80	**83.80**	61.60	95.20	**86.40**
1024	**LORE-KV /D**	46.26	86.60	99.60	**100.00**	75.80	96.80	99.45	99.40	99.40	**77.20**	83.60	**62.20**	**95.40**	86.29
						Context length = 16K									
128	CriticalKV	2.80	30.00	68.60	63.20	0.00	61.20	49.00	93.80	86.80	0.00	78.80	51.60	0.00	45.06
128	AnDPro	2.80	32.00	85.60	83.20	0.00	61.20	69.60	**100.00**	94.40	0.00	78.20	52.40	27.00	52.80
128	**LORE-KV /E**	**4.20**	**34.60**	**90.80**	88.40	**1.20**	**65.40**	74.00	**100.00**	**100.00**	1.20	**83.20**	56.00	29.60	**56.05**
128	**LORE-KV /D**	3.26	34.00	90.60	**89.00**	**1.20**	64.40	**74.25**	**100.00**	**100.00**	**1.60**	83.00	**56.60**	**29.80**	55.98
256	CriticalKV	7.60	53.00	75.00	77.00	**1.20**	**95.00**	80.40	95.60	91.20	1.00	79.00	52.20	49.00	58.25
256	AnDPro	7.60	54.20	86.00	90.00	**1.20**	**95.00**	**93.60**	**99.60**	96.20	1.00	78.40	52.80	66.60	63.25
256	**LORE-KV /E**	**14.80**	**70.20**	**86.20**	99.00	0.00	88.00	91.00	98.60	**96.60**	1.40	**79.80**	53.00	76.80	**65.80**
256	**LORE-KV /D**	13.86	69.60	86.00	**99.60**	0.00	87.00	91.25	98.60	**96.60**	**1.80**	79.60	**53.60**	**77.00**	65.73
512	CriticalKV	16.00	73.20	80.80	86.60	**24.20**	**98.20**	88.80	96.80	93.60	18.20	78.60	51.40	69.60	67.38
512	AnDPro	16.00	74.00	**87.20**	94.20	**24.20**	**98.20**	**96.40**	**99.00**	96.40	18.20	78.20	51.80	79.80	70.28
512	**LORE-KV /E**	**20.00**	**82.60**	**87.20**	98.80	23.20	94.40	95.20	98.40	**96.60**	18.20	**78.80**	51.80	85.60	**71.60**
512	**LORE-KV /D**	19.06	82.00	87.00	**99.40**	23.20	93.40	95.45	98.40	**96.60**	**18.60**	78.60	**52.40**	**85.80**	71.53
1024	CriticalKV	**61.20**	**84.20**	87.60	92.00	**65.40**	96.20	90.00	97.60	95.20	**74.40**	**81.40**	55.00	82.20	**81.72**
1024	AnDPro	24.80	75.80	**99.80**	99.40	57.80	**99.40**	**98.80**	**100.00**	**100.00**	51.00	80.80	55.80	88.40	79.37
1024	**LORE-KV /E**	27.20	81.00	**99.80**	**100.00**	57.40	97.00	98.00	99.60	**100.00**	51.00	81.20	55.80	91.80	79.98
1024	**LORE-KV /D**	26.26	80.40	99.60	**100.00**	57.40	96.00	98.25	99.60	**100.00**	51.40	81.00	**56.40**	**92.00**	79.87

Table 7; the corresponding Mistral code results in Table 1 are closer to neutral. A separate $M{=}1$, $T{=}8$ Llama run was completed only on NarrativeQA, Qasper, and MultiFieldQA-en and is therefore excluded from Table 7. Lookahead-count comparisons for Llama are instead based on the matched RULER ablation in Table 11.

D.3 RULER: Protocol and Per-Task Results

All RULER results are obtained under one matched implementation using chunk size 4, $w{=}32$, and $B \in \{128, 256, 512, 1024\}$; LORE-KV /E follows the core configuration in Table 5, and LORE-KV /D differs from it only in $(\gamma, \lambda_D) = (1, 0.3)$. Each task is scored on its native 0–100 metric, averaged across the 13 tasks at each context length and then across 4K, 8K, and 16K; Tables 8 and 9 report the complete per-task results. At tight budgets, LORE-KV /E gains are strongest on aggregation-style tasks such as VT, FWE, and CWE, while single-needle tasks become less discriminative as the budget increases. The main exception is the multi-query needle task, where LORE-KV /E can underperform AnDPro at shorter contexts, and NIAH-MK3 remains difficult for all methods at $B{=}128$. At the opposite extreme, CriticalKV leads the Llama $B{=}1024$, 16K setting (81.72 versus 79.98 for LORE-KV /E and 79.37 for AnDPro), which we report as an isolated exception to the overall trend.

D.4 RULER Ablations

Table 11 tests the main ablation axes on RULER, whose prompt structure differs from LongBench. The cleanest comparison is at 4K and $B{=}128$, where $M{=}1$ and the LORE-KV /E core use matched uniform head budgets and top-K selection: increasing M from 1 to 4 improves the average from 56.90 to 58.43. By comparison, moving γ from 1 to 1.3 changes the corresponding $M{=}4$ score from 58.39 to 58.43—a 23% perturbation along a single ray of the (β, γ) plane that never reaches $\gamma{=}0$, so it bounds the effect of that ray and not of the weighting apparatus, and the adaptive-budget and diversity switches each change the score by at most 0.11 in the available matched rows. At $B{=}256$, the available adaptive-budget comparison similarly improves from 68.50 with $M{=}1$ to 70.04 with $M{=}4$. Most variants were evaluated only at 4K. In particular, the available $M{=}1$ result at 16K uses adaptive rather than uniform head budgets, so the 52.18 versus 51.05 comparison is not a single-factor lookahead-count ablation.

Complete Configuration Grid Table 10 reports all executed LongBench configurations with Mistral-7B-Instruct-v0.2. It complements Table 10 by exposing the exact hyperparameter vector of every run; rows that change multiple switches are included for completeness but are not interpreted as single-factor ablations.

D.5 Needle-in-a-Haystack Grid

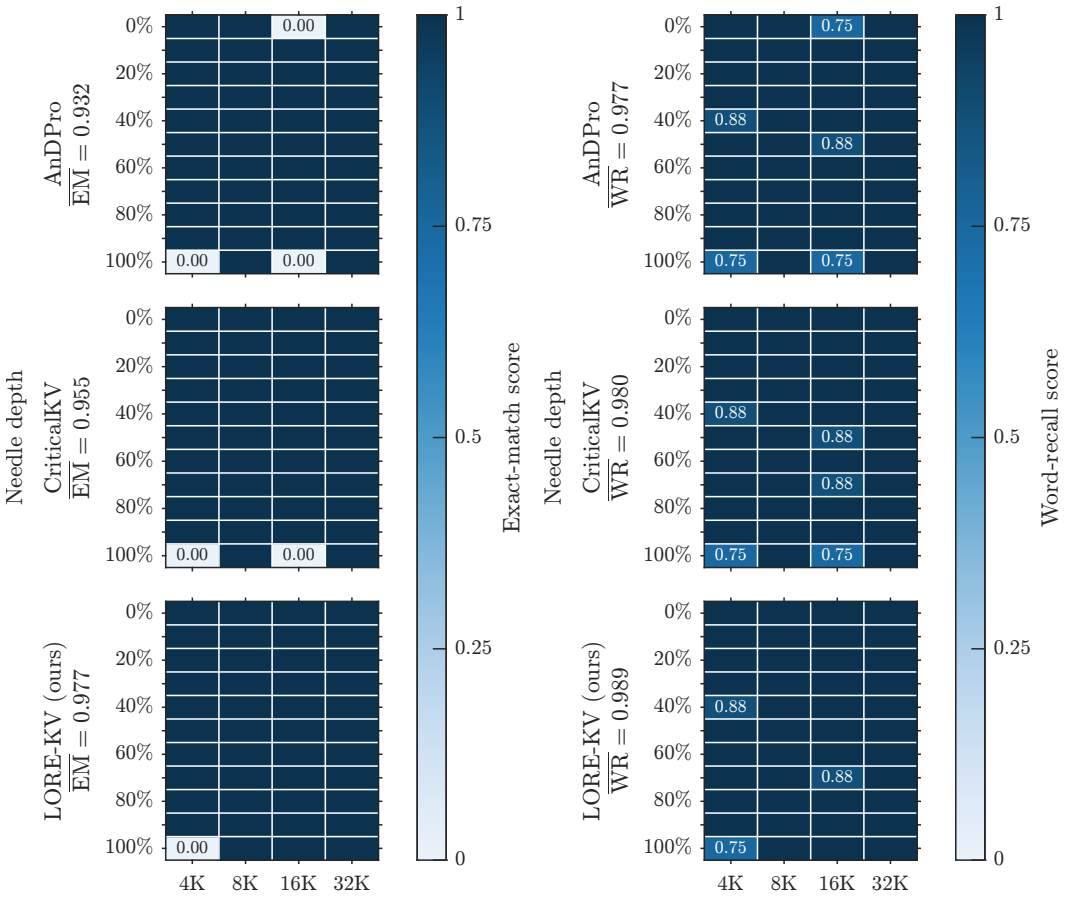
Figure 3 gives the complete depth×length grid for Llama-3.1-8B-Instruct at $B{=}1024$, with LORE-KV /E as the LORE-KV panel; the LORE-KV /D grid is pending. Retrieval is close to saturation at this budget, with differences concentrated at the more difficult depth–length combinations, so this setting has little power to separate the two reliability variants—which is why the LORE-KV /E-versus-LORE-KV /D comparison of Appendix F runs at $B{=}128$ instead. RULER provides the controlled low-budget counterpart used in the main analysis.

E Mechanism Analysis

E.1 Measurement Protocol

All mechanism analyses use a common capture procedure applied to the actual eviction implementation.

Captured states. For each prompt, we (i) run the standard prefill and retain the post-RoPE prompt-window queries and full $\mathbf{K}^{l,h}, \mathbf{V}^{l,h}$; (ii) sample the M lookahead trajectories and store their query states, the per-step centered score $c_u^{(m)}$, entropy $H_u^{(m)}$ and varentropy $V_u^{(m)}$, the raw and standardized trajectory disagreement $D_m, \hat{D}_m$, and the weight ω_m, with the three channels of Eq. equation 4



(a) Exact match **(b)** Word recall

Figure 3: **Needle-in-a-Haystack retrieval.** Exact-match performance across needle depths and 4K–32K context lengths with Llama-3.1-8B-Instruct, $B = 1024$.

Table 10: LongBench ablations (16-dataset average) with Mistral-7B-Instruct-v0.2. – denotes configurations not yet run.

Axis	Variant	B=128	B=256	B=512	B=1024
Core	LORE-KV /E (M=4, $(1, 1.3, 0)$)	40.28	41.38	41.86	42.20
	LORE-KV /D (M=4, $(1, 1, 0.3)$)	40.39	41.44	41.86	42.17
Lookahead budget	M=0 (prompt window only; λ_L=0)	35.87	37.50	38.70	39.50
	M=1 (single surrogate response)	39.81	40.85	41.56	42.03
	M=2	40.17	40.18	41.10	41.22
	M=8	40.69	40.97	41.52	41.65
Reliability weighting	Uniform weights, $(0, 0, 0)$	–	–	–	–
	Log-probability only, $(1, 1, 0)$	40.29	41.39	41.30	41.40
	Pure typicality, $(1, 0, 0)$	–	–	–	–
	Entropy + disagreement, $(1, 1.3, 0.3)$	40.33	41.40	41.84	42.17
	Disagreement without entropy, $(1, 0, 0.3)$	–	–	–	–
	Stronger entropy, $(1, 2, 0)$	–	–	–	–
Perturbation objective	Accumulated attention mass	–	–	–	–
	Unprojected norm, $\|\Delta \mathbf{o}\|_2^2$	–	–	–	–
	AnDPro scoring with the same continuations	–	–	–	–
Plug-ins	+ Adaptive head budgets	40.28	41.35	41.72	41.72
	+ Adaptive budgets and diversity	40.30	40.33	41.50	41.80

Configuration details. Unless otherwise stated, T=8, λ_L=1, uniform head budgets, and top-K selection are used. Adaptive head budgets use τ=1, while diversity adds λ_k=λ_v=0.1. Reliability weighting is vacuous at M=1, since the softmax over a single trajectory always gives unit weight. For M=2, $\hat{D}_m$ is identically zero by symmetry, so trajectory disagreement cannot affect the weights, although the typicality and entropy coefficients can. M=0 denotes the prompt-window-only configuration.

Table 11: RULER ablations with Mistral-7B-Instruct-v0.2 (13-task averages).

						B=128			B=256		
Configuration	M	γ	λ_D	**Head budgets**	**Selection**	4K	8K	16K	4K	8K	16K
AnDPro (strongest baseline)	–	–	–	–	–	54.85	50.06	45.20	71.05	64.08	57.91
M=1, uniform, top-K	1	1	0	uniform	top-K	56.90	54.29	52.15	68.45	62.77	57.70
M=1, adaptive, top-K	1	1	0	adaptive	top-K	56.92	54.31	52.18	68.50	62.81	57.75
M=1, adaptive, diversity	1	1	0	adaptive	diversity	56.95	54.34	52.20	68.55	62.85	57.78
M=4, $(1, 1, 0)$, uniform, top-K	4	1	0	uniform	top-K	58.39	55.72	50.98	70.08	64.45	58.85
M=4, $(1, 1, 0)$, adaptive, top-K	4	1	0	adaptive	top-K	58.28	55.68	50.95	70.04	64.42	58.82
M=4, $(1, 1, 0)$, adaptive, diversity	4	1	0	adaptive	diversity	58.31	55.70	50.97	70.07	64.45	58.85
M=4, adaptive, top-K	4	1.3	0	adaptive	top-K	58.37	55.74	51.02	71.45	65.46	59.88
M=4, **uniform, top-K (LORE-KV /E core)**	4	1.3	0	uniform	top-K	**58.40**	**55.77**	51.05	**71.51**	**65.52**	**59.94**
M=4, disagreement, adaptive, top-K	4	1	0.3	adaptive	top-K	58.28	55.70	50.96	69.72	64.50	58.92
M=4, disagreement, adaptive, diversity	4	1	0.3	adaptive	diversity	58.31	55.72	50.98	69.75	64.53	58.95
M=4, **uniform, top-K (LORE-KV /D core)**	4	1	0.3	uniform	top-K	58.31	55.74	51.00	69.77	64.56	58.98

stored separately; per-step storage costs MT floats per prompt; and (iii) greedy-decode against the *full* cache and record the post-RoPE queries at each decoding step. Disagreement is recorded even when λ_D=0, so that both reliability signals are available as diagnostics on every capture regardless of which variant produced it; the analysis code imports the same D_m routine the deployed method uses, so the figures and the runtime score identically by construction. Step (iii) defines $\mathcal{Q}_{\text{future}}$ and is used only for analysis. Queries are recomputed from each layer's own projection and rotary embedding, leaving the forward pass unchanged.

Analysis score. All token-importance analyses use the LORE-KV scoring rule. For query $\mathbf{q}$ and candidate token i, let $c_{q,i} = a_{q,i}^{l,h}/(1 - a_{q,i}^{l,h} + \epsilon)$. We use

$$s_{q,i}^{l,h} = |c_{q,i}| \left\| \mathbf{W}_{l,h}^{O} \mathbf{v}_i^{l,h} - \mathbf{W}_{l,h}^{O} \mathbf{o}_q^{l,h} \right\|_2 . \tag{19}$$

Table 12: Mechanism-analysis measurements, lookahead settings, and sample sizes.

Figure	Grouping variable	Measured quantity	Steps	M	Sample size (LB / RULER)
4	particles $M \in \{1, 2, 4, 8\}$	coverage, Eq. equation 20	32	8	118 / 200 prompts
5	candidate rank; criterion pair	score profile; overlap; Spearman ρ	0	4	4,501 / 3,843 candidates
1	cosine value; layer index	fidelity, Eq. equation 22	16	4	2.47M / 3.28M triples
6	binned confidence R_m; kept fraction	Recall @B; $1 - \mathrm{Cov}$	32	8	520 / 1,600 particles
2	mismatch $\mathrm{Mis}(X)$	recall and fidelity gain	24	4	265 / 200 examples
8	token class	retention; enrichment	0	4	751K / 769K tokens
9	layer; (layer, head)	energy E_l; recall gain $G_{l,h}$	24	4	32×32 grid
10	budget $B \in \{64, \ldots, 1024\}$	fidelity $F(B)$; implied saving	24	4	one score pass, five selections
7	disagreement D_m; entropy H_m; signal; weighting rule	Recall @B; Spearman ρ; coverage, Eq. equation 20	32	8	4,400 / 52,000 particles

Scores are averaged over the corresponding query set and combined using Eq. equation 8. Prompt-window queries retain their causal mask; lookahead and future queries occur after the full prompt and require none.

Compared criteria. All criteria use the same captured model state and selector. **Attention** is the mean SnapKV-style prompt-window attention mass; **AnDPro window** uses the projected perturbation with $\lambda_L{=}0$; **LORE-KV** uses Eq. equation 8 with the LORE-KV /E weights unless a panel states otherwise; and **Single-lookahead** combines the window with one unweighted trajectory as a score-level proxy for Lookahead Q-Cache. Protected positions $\mathcal{P}$ receive score $+\infty$, and the remaining tokens are selected by uniform per-head top-$\min(B, n)$. Thus, comparisons isolate the scoring criterion rather than downstream selection.

Analysis-only deviations. Four deviations from deployment apply to these figures. **D1:** Eq. equation 19 uses the magnitude rather than squared magnitude of Eq. equation 7; this preserves single-query rankings but changes the scale of the layer energy in Figure 9. **D2:** chunking, adaptive head budgets, and diversity are disabled, and B includes the $\sim$33 protected positions. **D3:** Figure 9 sums energy over all positions and displays GQA-expanded attention heads rather than KV heads. **D4:** Figure 8 identifies answer tokens by exact token-id match to the gold answer; this may underestimate absolute evidence retention but affects all criteria equally.

Derived quantities. Coverage measures how closely an estimated query set matches the true future queries:

$$\mathrm{Cov}\left(\mathcal{Q}_{\mathrm{future}}; \{\hat{\mathcal{Q}}_s\}, \boldsymbol{\pi}\right) = \frac{1}{H} \sum_h \frac{1}{T_{\mathrm{f}}} \sum_t \sum_s \pi_s \max_{\hat{\mathbf{q}} \in \hat{\mathcal{Q}}_s} \cos\left(\mathbf{q}_{t,h}^{\mathrm{future}}, \hat{\mathbf{q}}\right). \tag{20}$$

Queries are unit-normalized before computing cosine similarity. The oracle score $S^\star$ is Eq. equation 8 evaluated directly on $\mathcal{Q}_{\mathrm{future}}$, and

$$\mathrm{Recall}\,@B(S, S^\star) = \frac{|\mathrm{top}_B(S) \cap \mathrm{top}_B(S^\star)|}{B} \tag{21}$$

measures agreement with oracle retention. Behavioral fidelity evaluates the retained cache directly:

$$F(B) = \mathbb{E}_{\mathbf{q} \sim \mathcal{Q}_{\mathrm{future}}} \cos\left(\mathbf{o}_q^{\mathcal{R}(B)}, \mathbf{o}_q\right). \tag{22}$$

Finally, $\mathrm{Mis}(X) = 1 - \mathrm{Cov}(\mathcal{Q}_{\mathrm{future}}; \{\mathcal{Q}_{\mathcal{W}}^{l,h}\})$ denotes prompt-to-future query mismatch.

Configuration. Unless noted in Table 12, all mechanism analyses use Mistral-7B-Instruct-v0.2, the core configuration of Table 5 with the entropy weighting of LORE-KV /E, $B{=}128$, LongBench single-document QA, and RULER at 4K. The two reliability variants are compared explicitly only in Figures 4 and 7, which re-weight the captured particles under both rules; every other figure is insensitive to the choice, because it either fixes the weights or compares criteria that do not use them. Scatter plots may thin displayed markers, but all statistics use the complete samples.

E.2 Future-Query Coverage

Figure 4 compares the prompt-end window, one unweighted rollout, uniformly weighted multi-trajectory queries, and the two reliability-weighted estimators of Eq. equation 5 using Eq. equation 20. The M-sweep uses nested prefixes of the same eight-particle sample, so all multi-trajectory variants coincide at $M{=}1$. Because $\hat{D}_m$ is ensemble-relative, it is recomputed within each nested prefix rather than sliced from the full-ensemble value, which keeps the LORE-KV /D curve a valid

estimator at every M. Response-side queries produce the dominant coverage increase, while additional particles give diminishing returns after roughly $M{=}4$, and the two weighting rules separate only slightly—the ordering of the weights matters far less than the decision to use response-side queries at all. The smaller separation on RULER is consistent with its question-at-prompt-end format.

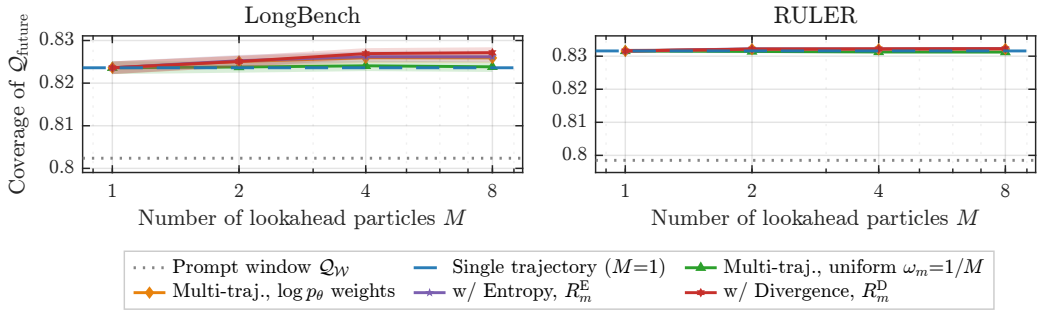


Figure 4: **Future-query coverage.** Prompt-window, single-trajectory, and multi-trajectory estimates as the number of lookahead particles increases. Bands show ± 1 s.e.m.

E.3 Retention-Order Shift

Figure 5 measures whether lookahead changes which tokens cross the retention boundary rather than merely rescaling prompt-side scores. At $B{=}128$, attention and LORE-KV /E have top-B overlap 0.82 on LongBench and 0.86 on RULER, despite high rank correlations ($\rho = 0.94$ and 0.87). Thus, most of the global ordering is shared while the tokens near the budget boundary differ. Single-lookahead and LORE-KV /E are substantially closer (0.97 overlap and $\rho = 1.00$ on LongBench; 0.92 and 0.95 on RULER), providing the score-level compute-matched comparison with a single response-side surrogate. The corresponding LORE-KV /E-versus-LORE-KV /D overlap on the same particles gates whether an end-to-end run can resolve the two variants at all.

E.4 Particle-Confidence Calibration

Figure 6 evaluates whether the combined confidence R_m, at the LORE-KV /E coefficients $(1, 1.3, 0)$, predicts oracle retention utility. Utility increases weakly with confidence, while hard removal of low-confidence particles provides little benefit: the residual coverage error is essentially flat in the kept fraction, and retaining only the most confident particles does not beat keeping all of them. This supports soft weighting rather than hard particle selection and is consistent with the small difference between $\gamma{=}1$ and the core $\gamma{=}1.3$ in the ablations—though, as Section 2.3 notes, that pair brackets a narrow range of one coefficient rather than the reachable set. Which individual signal carries this weak correlation—average log-probability, $-\bar{H}_m$, $-\hat{D}_m$, or either combined score—is measured in the third row of Figure 7, which ranks all of them against the same utility on one set of particles; the same figure supplies the entropy-versus-utility relationship itself, so it is not repeated here.

E.5 Trajectory Disagreement as the Reliability Signal

Figure 7 prices each channel of Eq. equation 4 against per-particle oracle utility. Every signal is recorded on the *same* captured particles, and the weighting rules in the last row re-weight that same ensemble, so the comparison carries no difference in test-time compute: whatever separates the rules is attributable to the coefficients alone. The capture uses $M{=}8$, since $\hat{D}_m$ is identically zero for $M \leq 2$, and the diagnostic forces every coefficient to be non-degenerate even when the run's own configuration set one to zero, so that the rules stay comparable. Note that $\bar{c}_m$ is recoverable from any existing capture with no re-decoding, since $\bar{c}_m$ equals the stored average log-probability plus the stored mean entropy; only the varentropy V_u of Eq. equation 13 requires a re-capture pass.

Two questions are answered in turn. **Is disagreement the better per-particle signal?** The first two rows plot each particle's oracle utility against D_m and against $\bar{H}_m$. Utility here is Recall $@B$ against

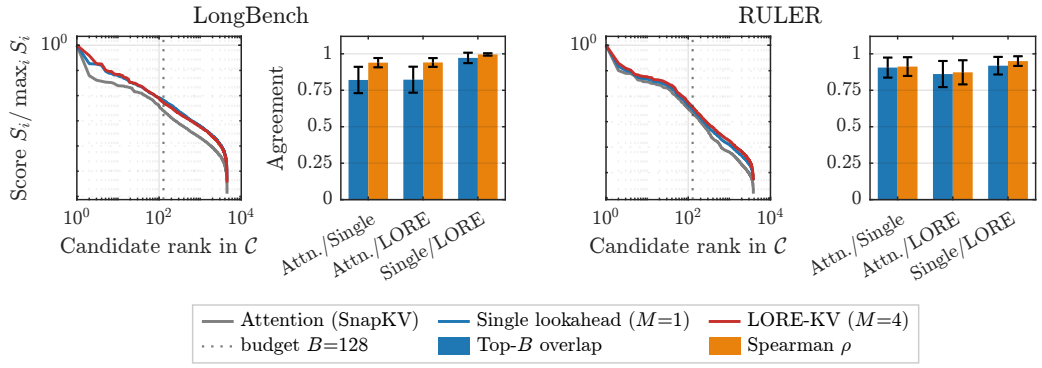


Figure 5: **Retention-order shift.** Candidate-score profiles, top-B overlap, and Spearman correlation across eviction criteria. Mistral-7B-Instruct-v0.2, $B = 128$, $M = 4$.

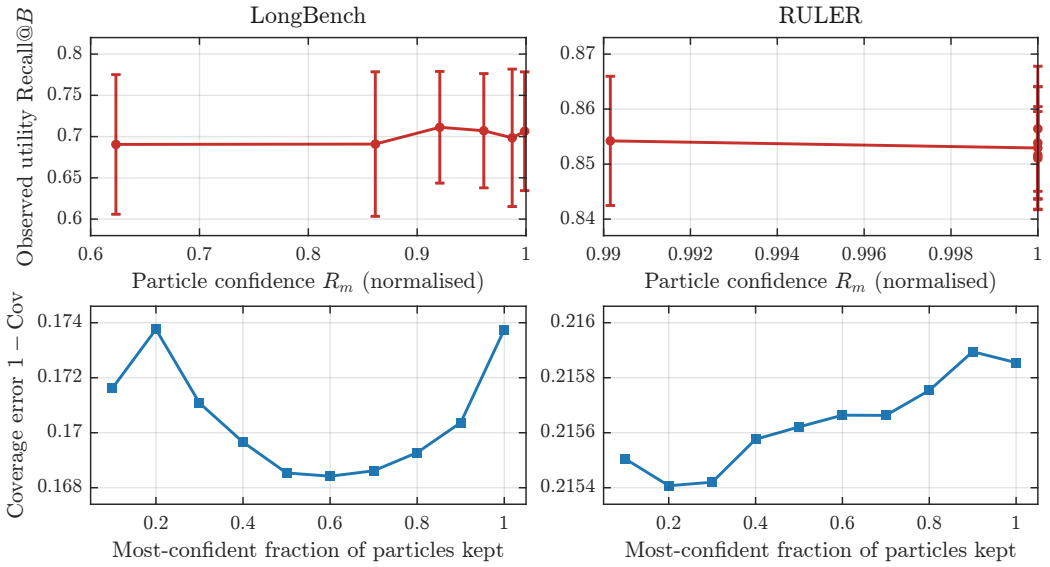


Figure 6: **Particle-confidence calibration.** Oracle retention recall versus reliability score (top) and residual query-coverage error after confidence filtering (bottom).

the oracle score $S^\star$ (Eq. equation 21), i.e. measured on the retention a single particle induces rather than on its queries alone; the capture also records each particle's standalone coverage of $\mathcal{Q}_{\text{future}}$ (Eq. equation 20) and the two orderings agree, so the figure reports the downstream quantity. A negative slope in the first row is the premise of the penalty: a trajectory the rest of the ensemble does not reproduce keeps tokens the true future queries would not have kept. The third row ranks every candidate signal by its Spearman correlation with the same utility, sign-oriented so that larger always means "better particle". Two comparisons matter there. The first is $\bar{c}_m$ against the raw $\log p_\theta$: if removing the entropy confound raises the correlation, the reparametrization improves the per-particle signal itself, measured on this paper's own diagnostic. The second is whether $-\hat{D}_m$ ranks above $-\bar{H}_m$, which would mean disagreement carries information entropy does not. On LongBench the correlations are – for $\log p_\theta$, – for $\bar{c}_m$, – for $-\bar{H}_m$ and – for $-\hat{D}_m$, with –, –, – and – on RULER. Because a marginal correlation cannot separate a signal from what it shares with the others, the *semipartial* correlation of D_m with the residual of utility on $(\bar{c}_m, \bar{H}_m)$ is what gates $\lambda_D > 0$ in the end-to-end grid.

Does the weighting built on it win? The last row reports ensemble coverage of $\mathcal{Q}_{\text{future}}$ under the $(\beta, \gamma, \lambda_D)$ cells $(0, 0, 0)$, $(1, 1, 0)$, $(1, 0, 0)$, $(1, 1.3, 0)$, $(1, 1, \lambda_D)$ and $(1, 0, \lambda_D)$—i.e. including the uniform and no-entropy controls that the earlier parametrization could not express. Consistent with

the calibration result of Figure 6—the reliability signal is real but weak—the absolute differences between the bars are small, which is exactly why the weighting stays soft and λ_D is bounded. The statistic to read is therefore not the bar height but the *paired* per-sample comparison printed in the panel, since absolute coverage varies far more across prompts than across weighting rules: the disagreement rule improves mean coverage by – and wins on –% of samples on LongBench, and by – on –% on RULER. This score-level evidence is available for both variants now; the corresponding end-to-end accuracies are the – rows of Tables 1, 3, 7, 10, 11 and 10.

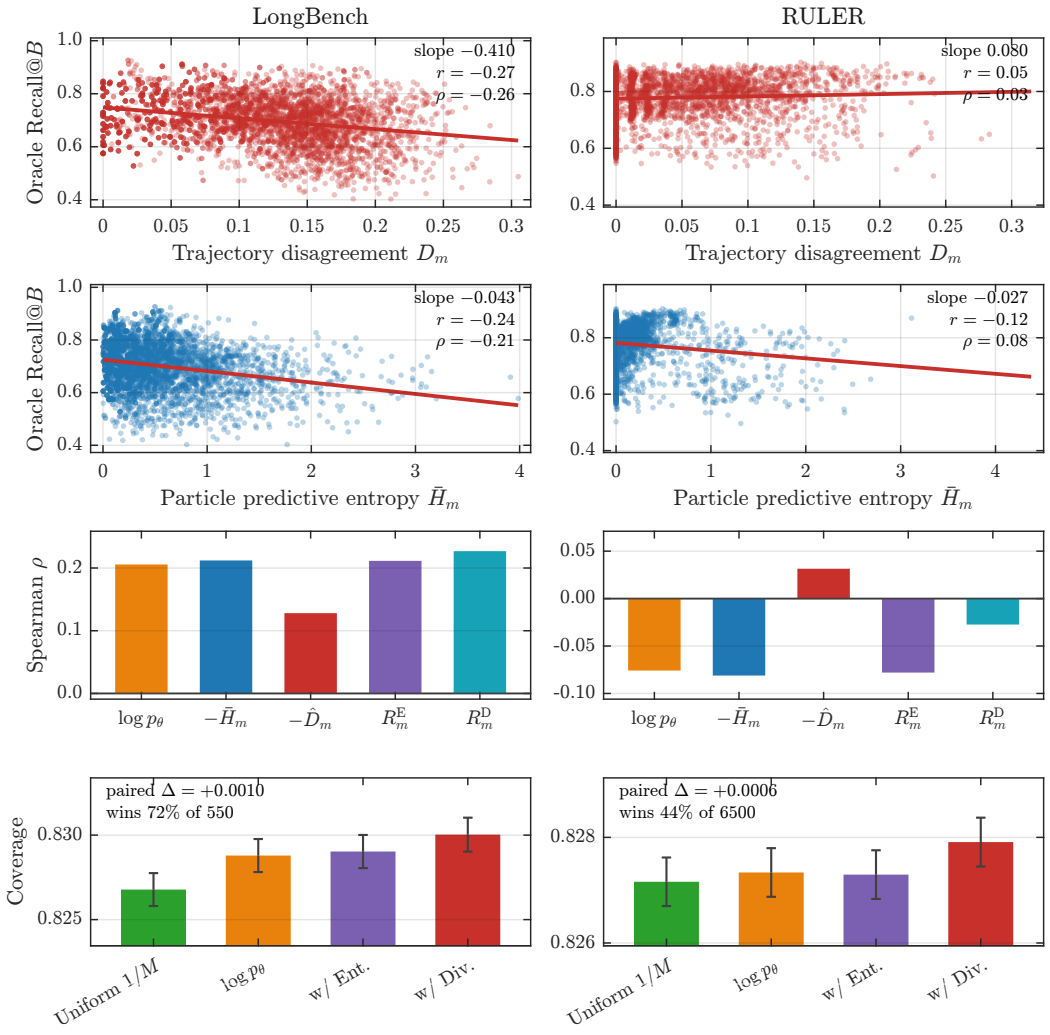


Figure 7: **Particle reliability signals.** Oracle retention utility versus trajectory disagreement and predictive entropy, signal correlations, and ensemble coverage under alternative weighting rules. All rules use identical particles. Mistral-7B-Instruct-v0.2, $B = 128$, $T = 8$, $M = 8$.

E.6 Retained-Token Composition

Figure 8 assigns each prompt token, in priority order, to attention sink, recency window, answer/evidence, number, punctuation/boilerplate, capitalized entity, or other. It reports both the fraction of heads retaining a token and the enrichment $P(c \mid \mathcal{R})/P(c \mid X)$.

The protected sink and recency classes are identical across methods. On LongBench, answer/evidence enrichment is 1.03 for attention, 1.00 for the AnDPro window, and 2.51 for LORE-KV /E; the corresponding retention rates are approximately 0.028, 0.028, and 0.069. On RULER, answer/evidence enrichment is 16.5, 16.3, and 20.3, respectively. Other token classes change only modestly, indicating that the main compositional difference is increased retention of answer-bearing

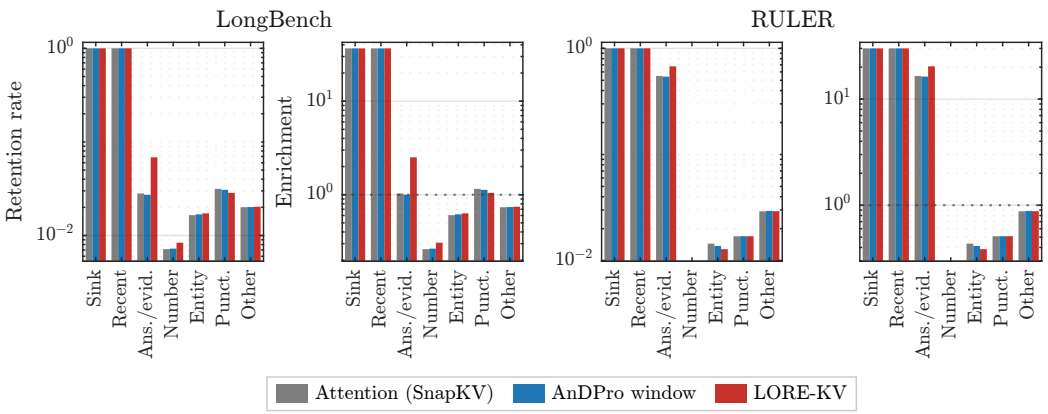


Figure 8: **Retention by token class.** Mean retained fraction (left) and class enrichment $P(c \mid \mathcal{R})/P(c \mid X)$ (right).

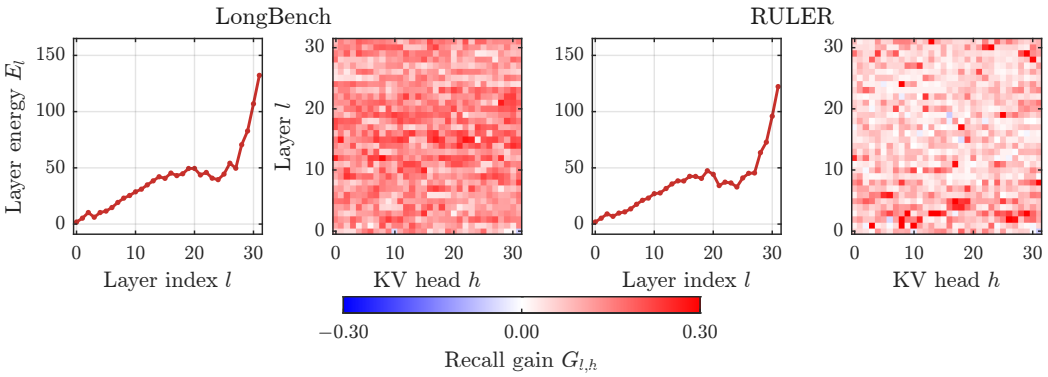


Figure 9: **Future-query signal across layers and heads.** Layer perturbation energy (left) and retention-recall gain from multi-lookahead queries (right). Mistral-7B-Instruct-v0.2, $B = 128$, $M = 4$.

evidence. Composition is a property of the retained set, so the LORE-KV /D counterpart is obtained by re-weighting the same captured particles under the disagreement rule and re-running selection only, with no additional decoding; it is not yet reported. This complements the retrieval analysis of Section 3.2.

E.7 Layer- and Head-Level Structure

Figure 9 localizes the perturbation signal across the network, under the LORE-KV /E weights of Appendix E.1. Layer perturbation energy increases with depth, while the gain $G_{l,h}$ from replacing window-only queries with multi-lookahead queries is positive for most heads but strongly non-uniform. The largest gains concentrate in a subset of middle-depth heads. This heterogeneity is consistent with the head specialization motivating head-wise retention methods (Feng et al., 2026; Fu et al., 2025) and provides the premise for the adaptive allocation rule in Eq. equation 17; its small end-to-end ablation gain indicates that most of the improvement is already captured by the core token score.

E.8 Budget Equivalence in Behavioral Fidelity

Figure 10 complements Table 2 by measuring cache quality continuously with the behavioral fidelity $F(B)$ of Eq. equation 22. On LongBench, LORE-KV /E increases from 0.852 at B=64 to 0.981 at B=1024, compared with $0.818 \to 0.975$ for attention and $0.818 \to 0.973$ for the AnDPro window.

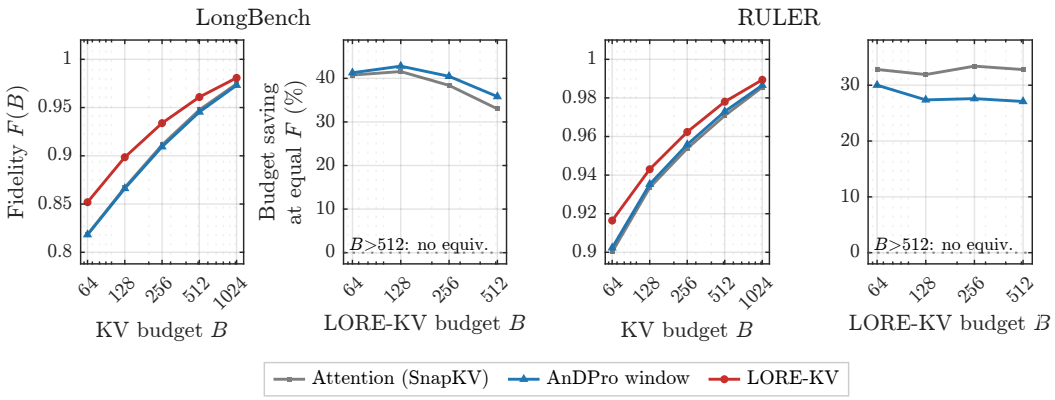


Figure 10: **Budget equivalence under behavioral fidelity.** Fidelity across cache budgets (left) and baseline cache savings required to match LORE-KV /E (right).

RULER shows the same ordering over a narrower range: $0.916 \rightarrow 0.989$ for LORE-KV /E versus $0.900 \rightarrow 0.986$ and $0.902 \rightarrow 0.987$ for the two prompt-side criteria.

Matching LORE-KV /E fidelity requires approximately 33–43% more retained-budget reduction on LongBench and 27–34% on RULER across the measured range. These values concern behavioral fidelity rather than task accuracy and therefore complement, rather than replace, Table 2. Where a baseline never reaches the target fidelity within the sweep, the corresponding saving is only lower-bounded.

F The Two Reliability Variants in the $(\beta, \gamma, \lambda_D)$ Plane

The paper carries two variants end to end, as points of the reparametrized plane: LORE-KV /E at $(\beta, \gamma, \lambda_D) = (1, 1.3, 0)$, whose LongBench averages with Mistral-7B-Instruct-v0.2 are 40.28, 41.38, 41.84 and 42.21 at $B = 128$, 256, 512 and 1024; and LORE-KV /D at $(1, 1, 0.3)$, whose end-to-end runs are –. The reparametrization is what makes the pair well-posed at all—in the earlier (λ_H, η) parametrization the two settings were not comparable coefficient-for-coefficient, and $\lambda_H{=}0$ silently carried a full unit of entropy penalty (Proposition 1). Read together, LORE-KV /D is not LORE-KV /E with the entropy channel swapped out: it keeps that channel at $\gamma{=}1$ and adds disagreement beside it, so the pair differs in *two* coefficients and is a variant comparison rather than a single-factor ablation. The single-factor cells that isolate one coefficient at a time—uniform $(0, 0, 0)$, pure typicality $(1, 0, 0)$, disagreement-without-entropy $(1, 0, 0.3)$ and stronger entropy $(1, 2, 0)$—are ablation rows of Table 10, not variants of the method, and are reported there. LORE-KV /D is calibrated per Section 2.3 so that $\lambda_D \hat{D}_m$ and $\gamma \bar{H}_m$ induce comparable realized spread.

Protocol. The comparison runs at $B{=}128$ only: Table 3 shows the criterion separating by 5.05 points at $B{=}128$ and 0.42 at $B{=}1024$, so the larger budgets have no resolving power for a weighting rule. Both variants re-weight nested prefixes of a *single* eight-particle draw per prompt, so neither differs from the other by test-time compute. We run three seeds minimum: the gaps at stake are 0.01 in Table 10 and 0.04 in Table 11, and the existing M-sweep is non-monotone ($M{=}2$ at 41.10/41.22 sits below $M{=}1$ at 41.56/42.03), so the noise floor is unknown and may well exceed them. If it does, "LORE-KV /D does not separate from LORE-KV /E" is the correct result and is what we will report.

G Limitations

LORE-KV trades one-time pre-generation computation for a better future-query reference. The lookahead trajectories are generated before decoding, preserving the same cache size and per-token decoding cost as other eviction methods, but adding noticeable wall-clock overhead. Unlike learned methods such as LookaheadKV, LORE-KV requires no offline training or model-specific learned parameters.

LORE-KV decides which KV pairs should be kept, but once a KV pair is removed, its information is lost. In contrast, methods such as IndexMem keep a compressed summary of evicted information (Yang et al., 2026), while LaProx considers interactions across heads and layers when selecting what to retain (Mai & Kim, 2026). LORE-KV also compresses the cache only once after the prompt, so extending it to update the cache during long or agentic generations is left for future work.

More broadly, future work should study how LORE-KV can preserve important instructions, structural information, and task-relevant evidence under aggressive compression, especially in multi-instruction, safety-sensitive, and long-context settings.